A Systems Perspective on Multi-Agent Applications: Programming, Evaluation, and Runtime

PANG, Wei

The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Large language model (LLM)-based multi-agent systems (MAS) have emerged as a powerful paradigm for solving complex tasks through inter-agent collaboration. Existing research typically addresses only a narrow slice of the design space: some frameworks focus on programming abstractions without runtime optimization; some optimize execution but assume fixed task structure; still others propose evaluation metrics for specific domains without connecting them to underlying agent architecture. What is missing is a *systems* perspective that unifies programming, evaluation, and runtime concerns into a coherent stack.

This thesis develops a three-layer *agentic systems stack* and presents four contributions situated within it. First, I propose AutoGen, a conversation-programming framework that unifies LLM inference, tool invocation, and human input under a `ConversableAgent` abstraction with a common send/receive/generate-reply interface, and a two-step development workflow based on conversation programming. Second, I introduce Paper2Poster, a real-world multi-agent workload that converts a $\sim$23-page scientific paper into a single-page poster, together with a four-dimensional evaluation framework (visual, textual, holistic, quiz-based) and the PosterAgent reference pipeline. Third, I propose DynTaskMAS, a dynamic task-graph–driven runtime that decomposes tasks into dependency-aware directed acyclic graphs, schedules them across parallel agents with priority-based queuing, and manages shared context through semantic-aware distribution, delivering 21–33% execution-time reductions and near-linear scaling up to 16 concurrent agents. Fourth, through a cross-layer synthesis I identify the missing compilation pathway from conversation programs to runtime task graphs as the principal open interface in the stack, and articulate four additional open challenges across the three layers.

The ultimate goal of this thesis is to advance the construction of large-scale multi-agent language-model applications through a coherent, cross-layer systems treatment.

Contents

Abstract **iii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Three Research Questions . . . 1
1.3 The Agentic Systems Stack . . . 2
1.4 Thesis Contributions . . . 2
1.5 Methodological Posture . . . 3
1.6 Thesis Outline . . . 3

2 Background and Related Work **5**
2.1 LLMs and Agents . . . 5
2.2 Programming Models and Frameworks . . . 5
2.3 Benchmarks and Evaluation . . . 6
2.4 Runtime and Scheduling . . . 6
2.5 Existing Surveys . . . 7
2.6 Summary . . . 7

3 Programming Model: Conversation-Programmed Agents **9**
3.1 Motivation . . . 9
3.2 The Conversable-Agent Abstraction . . . 9
3.3 Conversation Programming . . . 10
3.4 Applications . . . 13
3.5 Discussion . . . 16
3.6 Summary . . . 17

4 Workload and Evaluation: Multi-Modal Poster Generation **19**
4.1 Motivation . . . 19
4.2 Task Definition and Benchmark . . . 19
4.3 Four-Dimensional Evaluation . . . 20
4.4 PosterAgent Pipeline . . . 23
4.5 Results . . . 25
4.6 Summary . . . 28

5 Runtime: Dynamic Task-Graph Scheduling **29**
5.1 Motivation . . . 29
5.2 Architecture Overview . . . 29
5.3 Formalisms and Scheduling Algorithm . . . 30
5.4 Results . . . 33
5.5 Discussion . . . 35
5.6 Summary . . . 37

6 Cross-Layer Synthesis and Open Challenges **39**
6.1 Connecting the Layers . . . 39
6.2 Open Challenges . . . 40

6.3 Summary 43
7 Conclusion and Future Work **45**
7.1 Summary of Contributions 45
7.2 Future Work 46

List of Figures

1.1 The three-layer agentic systems stack. 2

3.1 AutoGen agent hierarchy. 10
3.2 A minimal two-agent AutoGen program. 12

4.1 Paper-to-poster compression ratios across four dimensions. 21
4.2 PosterAgent pipeline. 25
4.3 PaperQuiz score versus cost per poster. 26

5.1 DynTaskMAS architecture. 30
5.2 DynTaskMAS throughput scaling. 35

6.1 Concrete data flows across the three layers. 40

List of Tables

3.1 Positioning of the conversation-programming paradigm against contemporary multi-agent frameworks. 13
3.2 Summary of the six application scenarios. 15

4.1 Paper2Poster evaluation dimensions. 23
4.2 Poster generation methods comparison. 26

5.1 DynTaskMAS performance: execution time and scalability. 34
5.2 Travel-planning case study: seven domain-specialized agents, before and after DynTaskMAS. 35

6.1 Cross-layer concept mapping. 40

Chapter 1

Introduction

1.1 Motivation

The rapid progress of large language models (LLMs)—including GPT-4 [23], LLaMA [29], and Claude [1]—has catalyzed a fundamental shift in how autonomous software agents are designed. Where earlier agent systems relied on hand-crafted rules or narrow reinforcement-learning policies, modern *LLM-based agents* leverage the broad world knowledge, instruction-following ability, and code-generation capacity of foundation models to tackle open-ended tasks with minimal task-specific engineering [31].

A natural next step—and one that the community has pursued vigorously—is to compose *multiple* such agents into collaborative teams. The resulting landscape is rich and rapidly growing: multi-agent debate, role-playing societies, software engineering pipelines, task-decomposition systems, and orchestration frameworks now number in the dozens [9, 31] (Chapter 2).

Despite this rapid proliferation, existing frameworks specialize: some emphasize programming abstractions but leave execution unoptimized; others focus on execution but hold task structure fixed; others still advance domain-specific evaluation metrics with no coupling to the underlying architecture. The practitioner pays for this specialization by manually stitching programming models, evaluation harnesses, and execution engines together, often with brittle glue code and ad hoc performance tuning.

What is largely missing from the current discourse, however, is a *systems* perspective. Building a reliable multi-agent application involves far more than defining agent roles: one must choose programming abstractions that allow flexible composition, design evaluation protocols that capture the unique demands of multi-modal, long-context workloads, and engineer a runtime that schedules heterogeneous agents efficiently. These concerns mirror the classical layered-systems architecture that underpins modern computing: a programming language defines how computations are expressed; benchmarks and workloads characterize what the system must do; and an operating-system scheduler decides how and when work is executed.

The analogy is more than superficial. Just as the success of modern software depends on the co-design of programming languages (C, Python), workload suites (SPEC, MLPerf), and schedulers (Linux CFS at the OS level, Kubernetes at the cluster level), the success of multi-agent AI applications will depend on coherent choices at each layer and well-defined interfaces between them. A high-level agent programming model that cannot express parallelism leaves performance on the table; an evaluation framework that ignores computational cost gives misleading quality signals; and a runtime scheduler that is unaware of agent semantics cannot perform intelligent context routing.

1.2 Three Research Questions

To structure this investigation, I organize the thesis around three questions that any practical MAS deployment must answer:

1. **How do we *program* multi-agent systems?** What abstractions allow developers to define agent capabilities, wire up communication topologies, and mix human and machine control?

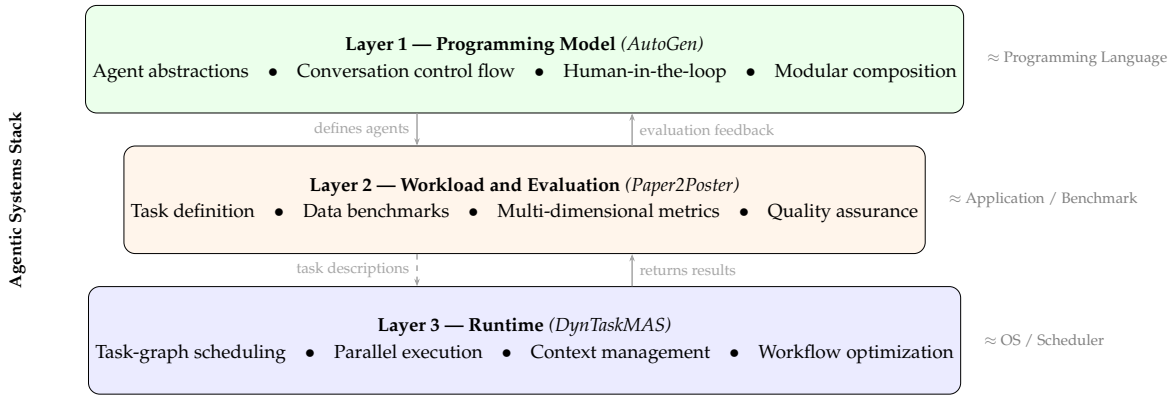

Figure 1.1: The three-layer agentic systems stack. Layer 1 provides programming abstractions (AutoGen), Layer 2 defines workload and evaluation (Paper2Poster), and Layer 3 supplies the runtime (DynTaskMAS). Solid arrows show existing information flow; the dashed arrow indicates an interface that remains an open challenge (Chapter 6). The classical computing analogy is annotated on the right.

2. **What does the *workload* look like?** What are the computational, multi-modal, and evaluative demands of a realistic multi-agent task, and how do we measure quality?

3. **How do we *run* multi-agent systems efficiently?** How can a runtime exploit parallelism, manage shared context, and adapt to dynamic task graphs?

1.3 The Agentic Systems Stack

I argue that these three questions correspond to three layers of an *agentic systems stack* (Figure 1.1): Layer 1 is AutoGen, a conversation-programming model; Layer 2 is Paper2Poster, a multi-modal workload with a four-dimensional evaluation framework; Layer 3 is DynTaskMAS, a dynamic task-graph runtime. Each layer is the subject of a dedicated chapter (Chapters 3, 4, 5) in which I describe the design and results; the next section enumerates the resulting thesis contributions in detail.

1.4 Thesis Contributions

This thesis makes four contributions.

C1 — Programming Model. I propose the conversation-programming paradigm and the *conversable agent* abstraction (Chapter 3). A single `send`/`receive`/`generate_reply` interface unifies LLM inference, tool invocation, and human input, and a two-step development workflow (define agents, program their interaction) replaces rigid hand-coded orchestration.

C2 — Workload and Evaluation. I introduce Paper2Poster (Chapter 4), a challenging real-world multi-agent workload that converts a ∼23-page scientific paper into a single-page poster, together with a four-dimensional evaluation framework (visual, textual, holistic, quiz-based) and the PosterAgent reference pipeline.

C3 — Runtime. I propose DynTaskMAS (Chapter 5), a dynamic task-graph–driven runtime that decomposes tasks into dependency-aware DAGs, schedules them across parallel agents with priority-based queuing, and manages shared context through semantic-aware distribution.

C4 — Cross-Layer Synthesis. Through a cross-layer analysis (Chapter 6) I identify the missing compilation pathway from conversation programs to runtime task graphs as the principal open interface in the stack, and articulate four additional open challenges across the three layers.

1.5 Methodological Posture

A thesis that claims to develop a *systems* stack rather than three disconnected contributions takes on a burden of proof the usual paper-by-paper publication cycle does not. I want to say briefly, before the chapters themselves, what methodological stance I adopt for meeting that burden.

First, each layer is demonstrated through a concrete working artifact rather than through formal characterisation alone. The programming model of Chapter 3 is grounded in a released Python library that real users can (and do) compose; the workload of Chapter 4 is grounded in a 100-pair benchmark with reproducible evaluation scripts; the runtime of Chapter 5 is grounded in a measured implementation on a specific four-GPU cluster. Artifacts matter because the questions the thesis engages with—what abstractions help practitioners, what metrics predict deployment quality, what runtime structure delivers actual speed-ups—are empirical in character. A formal-only treatment would answer different questions.

Second, I privilege *cross-layer consistency* over per-layer depth. A deeper programming-model chapter might enumerate more design variants; a deeper workload chapter might report more benchmarks; a deeper runtime chapter might prove more scheduling properties. In each case I have deliberately stopped at the depth required to substantiate the chapter's contribution to the stack as a whole. The stack is the thesis-level claim; the chapters exist to support it.

Third, the cross-layer synthesis of Chapter 6 is not a summary of the preceding chapters but a separate analytical contribution. It identifies the specific interfaces between the layers, names the gap that none of the three layers closes on its own (the programming-to-runtime compilation pathway), and articulates the research program that follows. A reader primarily interested in the stack's design space is encouraged to read Chapter 6 even after the three technical chapters.

1.6 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 reviews related work across the three layers, situating each of the thesis's contributions against the specific frameworks, benchmarks, and runtimes that come closest to it. Chapter 3 presents the programming-model layer: I propose the conversable-agent abstraction and the conversation-programming paradigm, develop six applications that stress different conversation patterns, and position the framework against LangChain, CAMEL, MetaGPT, and BabyAGI. Chapter 4 analyses the workload-and-evaluation layer: I introduce Paper2Poster as a canonical multi-agent workload, a four-dimensional evaluation framework, and the PosterAgent reference pipeline; I report quantitative results against four baselines and analyse where each baseline falls short. Chapter 5 describes the runtime layer: I propose DynTaskMAS, a dynamic task-graph–driven runtime with priority-based scheduling and semantic-aware context management; I evaluate execution-time reductions and near-linear scaling on synthetic and case-study workloads; and I present a concrete pseudocode for the APEE main loop. Chapter 6 offers the cross-layer synthesis: I identify four inter-layer interfaces, sketch a concrete design for the missing programming-to-runtime compilation

pathway, and articulate five open challenges for the community. Chapter 7 concludes and sketches future work, mapping the five challenges onto immediate research lines.

Chapter 2

Background and Related Work

This chapter situates the three layer-specific contributions of this thesis (C1–C3) against prior work across the three layers of the agentic systems stack, and positions the cross-layer synthesis (C4) relative to existing surveys.

2.1 LLMs and Agents

Chapter 1 motivated this thesis against the backdrop of the LLM-based agent explosion driven by foundation models [1, 23, 29, 31] and the subsequent wave of multi-agent composition [9, 31].

Two short observations situate the literature I discuss below. First, the gap between a *capable* LLM and a *useful* autonomous agent turns out to be filled by system design rather than by model scaling. A larger GPT-*n* does not obviate the need for a programming-model abstraction, a benchmark with measurable quality, or a runtime that manages parallelism and shared context—if anything, as models become cheaper and more available, the engineering cost of composing them dominates the per-call compute cost, and the system-design pressure grows. Second, the literature is strikingly disconnected across the three layers. Papers proposing new programming models rarely report runtime throughput numbers; papers proposing new runtimes rarely compare against a rich multi-dimensional evaluation; papers proposing new benchmarks rarely articulate what programming-model features would be needed to improve on them. This disconnection is not just a stylistic observation—it makes it hard for a practitioner to choose a consistent stack.

The remainder of this chapter organizes that literature along the three layers of the agentic systems stack: programming models and frameworks (§ 2.2), benchmarks and evaluation (§ 2.3), and runtime and scheduling (§ 2.4); § 2.5 positions the thesis against earlier survey work.

2.2 Programming Models and Frameworks

The earliest LLM-based multi-agent efforts focused on defining agent roles and communication patterns. CAMEL [17] introduces *inception prompting*, where two agents role-play as an AI user and an AI assistant to collaboratively solve tasks—pioneering the idea that inter-agent conversation itself can drive problem solving. MetaGPT [11] takes a software-engineering perspective, assigning each agent a specialized role (product manager, architect, project manager, engineer, QA engineer) and enforcing a structured standard operating procedure (SOP) to coordinate outputs. BabyAGI [21] demonstrates autonomous task decomposition and prioritization using a single LLM that continuously generates subtasks. On the orchestration side, LangChain [4] provides modular tooling for chaining LLM calls with external tools, and ReAct [32] interleaves reasoning traces with action steps to improve grounding. AutoGen, the framework I present in Chapter 3, differs from these by proposing a *unified* conversable-agent abstraction that subsumes LLMs, humans, and tools under a single interface, with programmable conversation control flow that supports sequential, nested, and group topologies.[1]

[1] LangChain has since grown a graph-based extension, LangGraph [15]; the AutoGen framework I describe here corresponds to the design I present in Chapter 3 rather than subsequent iterations (v0.4+ [20]) that added

2.3 Benchmarks and Evaluation

Existing multi-agent benchmarks fall broadly into two classes. The first is *capability-targeting*: each benchmark isolates a single skill and asks whether the agent (or agent team) succeeds on that axis. MATH [10], with 12,500 competition problems stratified by difficulty, measures step-by-step mathematical reasoning and produces a single accuracy number per system; ALFWorld [28], with 6 task types and 134 unseen test tasks, measures embodied decision-making under a partial-observability interface; Natural Questions [14] (with its 300k training queries backing smaller task-specific subsets) measures open-domain Q&A with a retrieval-then-read structure. Each is internally valuable and widely used. None of them, however, rewards the capabilities that distinguish a real multi-agent system from a well-tuned single agent: sustained multi-modal reasoning across tens of thousands of tokens, quality judged by multiple simultaneous criteria, or the iterative refinement that arises naturally when a Commenter can critique a Painter.

The second class is *task-targeting*: each benchmark picks a real application, defines ground truth, and measures how well a system solves that application. In the document-generation space POSTERSUM [27] collected 16,305 paper–poster pairs from ICML, ICLR, and NeurIPS 2022–2024 for scientific-poster summarisation, and PPTAgent [33] addresses slide generation with a dedicated evaluation harness. These task-targeted benchmarks are closer to the multi-agent evaluation needs of this thesis, but each still reports scores on a single quality axis (summary similarity, slide quality) rather than separating the dimensions that system design actually trades off. Paper2Poster, which I present in Chapter 4, inherits POSTERSUM's test split and task framing and goes further by defining a comprehensive four-dimensional evaluation framework (visual quality, textual coherence, holistic VLM-as-Judge assessment, and a novel quiz-based information-transfer probe) that treats poster generation as a full-stack multi-agent stress test and that explicitly surfaces the multi-objective trade-off structure on which system choices actually depend.

2.4 Runtime and Scheduling

Most existing frameworks execute agent interactions synchronously. LangChain processes chains step by step; AutoGen's `generate_reply()` event loop handles one message at a time; and MetaGPT's SOP is inherently sequential. LangGraph [15] adds graph-based state machines that enable conditional branching and cycles, which is an important step because it turns a linear chain into a topology where branches are explicit, but it still does not automatically parallelize independent subtasks—the graph node transitions are evaluated one at a time, and any parallelism the user wants must be orchestrated by hand. The distance between "the framework permits parallelism" and "the framework exploits parallelism" is where runtime-level contributions begin.

Classical scheduling literature offers a mature vocabulary for the missing half. Directed-acyclic-graph (DAG) scheduling for parallel dataflow has been studied since the 1980s, with variants such as list scheduling, critical-path scheduling, and HLFET/HEFT producing near-optimal makespans on heterogeneous processors; distributed shared-memory systems have similarly developed tag-based coherence and semantic-similarity routing for large-scale data distribution. The core problem these techniques solve—extracting parallelism from a task-dependency graph whose nodes are the unit of work—is structurally identical to the problem an LLM-based multi-agent runtime faces, with the important wrinkle that LLM agents have non-deterministic execution times and dynamically evolving task

graph-based and distributed execution features outside the scope of this thesis.

structures. DynTaskMAS, the runtime I present in Chapter 5, addresses the gap with dynamic task-graph construction, priority-based parallel scheduling, and semantic-aware context management—drawing on this classical literature while adapting each technique to the peculiarities of LLM-backed execution.

2.5 Existing Surveys

Guo et al. [9] categorize multi-agent systems by agent profiling, perception, and action modules, presenting a taxonomy that follows the internal structure of individual agents rather than the interactions between them. Their survey is valuable for anyone building a new agent: it organises the literature on capabilities by what each capability enables an agent to do in isolation. Wang et al. [31] complement this by surveying autonomous agent architectures with an emphasis on memory, planning, and tool use—components that cross the agent boundary but still live within a single agent's responsibilities.

Both provide valuable domain-level taxonomies but do not address the *systems* concerns—programming abstractions, workload characterization, runtime optimization—that arise when building and deploying multi-agent applications at scale. A practitioner reading either survey comes away with a rich picture of what an individual agent can be made to do; a much sparser picture of how agents compose into a stack; and essentially no discussion of the run-time concerns that dominate deployment cost. This thesis is complementary rather than competitive: I borrow the individual-agent taxonomies where helpful (for example, the capability categorisation in Chapter 3's `ConversableAgent` discussion echoes the LLM/human/tool split that both surveys recognise) and layer the systems view on top.

A second category of related work worth naming is *application-domain-specific* surveys—for example, surveys of LLM-based code-generation systems or of embodied-AI assistants. These are even narrower in scope than the general-agent surveys above and do not attempt cross-layer analysis. Their value to a reader of this thesis is as evidence of the diversity of application domains that would benefit from a common systems substrate; I treat them as such rather than as direct alternatives. My contribution is not a comprehensive catalogue of all MAS work, but three layer-specific contributions that together span the systems stack plus a cross-layer synthesis that identifies the interfaces between them, using the programming/workload/runtime layering to reveal how choices at one layer constrain and enable options at others.

2.6 Summary

The existing literature covers programming abstractions, benchmarks, and execution runtimes in isolation, but rarely across layers. This fragmentation motivates the systems-layering treatment I pursue in Chapters 3–5, and the cross-layer synthesis in Chapter 6.

Chapter 3

Programming Model: Conversation-Programmed Agents

3.1 Motivation

In this chapter I present the programming-model layer of my agentic systems stack: AutoGen, a framework that enables development of LLM applications through multi-agent conversation. AutoGen introduces a *conversation-programming* paradigm in which both computation and control flow are expressed via message exchanges among *conversable agents*. It provides the foundational abstractions—agent types, communication interfaces, and control-flow mechanisms—on which the higher layers of this thesis build.

The key insight behind the framework is that the conventional approach of defining agent workflows through explicit control logic (if-then-else chains, state machines, DAG specifications) is unnecessarily rigid. Three observations motivate the departure from explicit control logic. First, the workflow of a realistic agent system is rarely known in advance: the conversation branches depending on what an LLM returns, what a human inputs, and what a tool call produces, and trying to enumerate those branches statically either over-restricts the system (each unforeseen branch becomes a bug) or produces control logic whose complexity rivals that of the work being automated. Second, different workflow stages call for different control primitives—some are best expressed by a developer writing Python ("if token count exceeds threshold, truncate"), some by an LLM interpreting natural language ("if the previous reply is incomplete, ask for elaboration"), and some by a hybrid that begins one way and continues the other. Explicit control logic privileges the first primitive at the expense of the others. Third, the natural unit of progress in a multi-agent system is a *message*, not a control-flow step: agents communicate by sending and receiving, and the conversation state is precisely the message log.

Instead, AutoGen treats *conversation itself* as the programming medium: agents exchange messages, and the conversation's natural progression—guided by a combination of natural-language instructions, programmatic constraints, and LLM decisions—determines the workflow. The resulting workflow blends programming and natural language at both the design and runtime stages. Developing an application becomes a two-phase activity: first, picking and configuring the agents that will participate (each with its role, its capability backends, and its human-input mode); second, describing how they should interact, with conversation messages themselves as the primary control surface. The first step is structural and typically short: an application might instantiate two or three built-in agents, perhaps wrap a tool function, and stop there. The second step is where the expressive freedom lives: custom reply functions, termination conditions, speaker-selection strategies, and human-input modes all parameterise the same underlying send/receive/generate-reply loop, and combining them yields a rich space of workflows without any extra control-flow scaffolding.

3.2 The Conversable-Agent Abstraction

Conversable Agents. The central type in my framework is `ConversableAgent`: a role-playing entity whose only externally visible actions are sending and receiving messages, and whose internal state is the accumulated history of the conversation it has participated in.

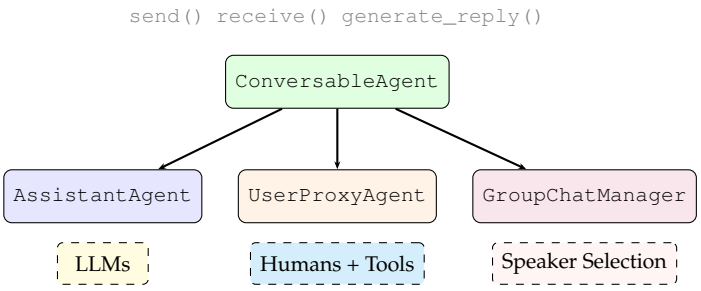

Figure 3.1: AutoGen agent hierarchy. All agents inherit from the `ConversableAgent` base class and share the unified `send/receive/generate_reply` interface.

Any concrete capability that an agent exposes comes from attaching one or more *capability sources* to the agent:

- an **LLM backend**, which handles role play, implicit state tracking, feedback generation, and code synthesis;
- a **human backend**, whose level of involvement is set by a mode parameter (always consulted, never consulted, or consulted conditionally);
- a **tool backend**, which handles code execution and function calls into external systems.

The three capability sources are not mutually exclusive; a single agent can mix them, and the mix-and-match property is what lets the same abstraction model an AI assistant, a tool-executing middleman, a human-in-the-loop operator, or any combination.

All inter-agent interaction goes through three methods: `send(msg, recipient)`, `receive(msg, sender)`, and `generate_reply(context)`. The last is where the work happens: it takes the incoming message together with the agent's running context and produces the agent's next utterance—via an LLM call, a tool invocation, a human consultation, or a custom reply function registered by the developer. The key design choice is that the *interface* to all these reply paths is identical; an agent backed by an LLM and an agent backed by a human are interchangeable from the caller's perspective.

Three pre-configured subclasses ship with the framework (Figure 3.1). `AssistantAgent` is an LLM-backed agent—GPT-4 in my running examples—preset with a system prompt that suits generic code-driven problem solving, and with its human-input mode turned off. `UserProxyAgent` is its complement: a human- and tool-backed agent that acts as the user's deputy, soliciting human input and executing code when needed. `GroupChatManager` is a coordinator rather than a worker: it orchestrates a roomful of agents by picking who speaks next, collecting that agent's response, and relaying it to the others.

3.3 Conversation Programming

AutoGen frames multi-agent development through two concepts:

1. **Computation**: the actions agents take to produce replies (LLM inference, code execution, human consultation).
2. **Control flow**: the sequence and conditions under which these computations occur.

Control flow is realized through three modes:

- *Natural-language control*: prompt-based instructions steer agent behaviour through the LLM's instruction-following ability. The default `AssistantAgent` system prompt, for instance, asks the agent to emit a distinctive termination token when the task is finished, describes the error-recovery protocol in English (retry after noting what failed), and specifies the structural shape the output should take so that downstream tools can parse it. The prompt combines five distinct prompting techniques: role play, control-flow directives, output confinement, automation facilitation, and grounding cues.
- *Programming-language control*: Python code specifies termination conditions, human-input modes, maximum auto-reply counts, and tool-execution logic. For instance, a developer can set `max_consecutive_auto_reply=10` to cap the number of turns, or define a custom termination function that checks whether a specific condition is met in the conversation.
- *Hybrid control*: transitions between the two, enabling the strengths of each to complement the other. An LLM can propose a function call that transfers control to programmatic logic (e.g., invoking a database query), and conversely, a custom reply function can invoke LLM inference with specific prompts to make a natural-language control decision. This bidirectional transition makes the control model more expressive in practice than either pure-NL or pure-code approaches alone.

Execution on top of the three primitives is driven by an **auto-reply event loop**. The loop's rule is simple: an agent that receives a message calls `generate_reply()` on its current context, sends the result as an outbound message, and waits for the next incoming message—unless a termination predicate fires first. Because `generate_reply()` itself is the composition of a priority-ordered chain of reply functions, the agent's behaviour at any given turn is determined by whichever registered reply function is the first to produce a non-`None` answer.

The default reply chain covers the common cases: an LLM-inference reply for turns that need an LLM, a code-or-function-execution reply for turns that dispatch a tool call, and a human-input reply for turns that solicit a user. Developers override or augment this by calling `register_reply()` to prepend custom functions to the chain; the semantics remain the same, only the first hit wins. The effect at the level of the whole application is that workflow orchestration becomes distributed—each agent decides locally how to respond to messages directed at it, and the conversation state is exactly the sequence of those responses. Nothing else acts as a central control plane.

Integration of natural-language and programmatic control. The three control modes above are not parallel options; they form a lattice of increasing expressiveness. Natural-language control alone is appropriate for agents whose only role is to interpret the intent of a prompt and emit an output. Programming-language control alone is appropriate for agents whose job is to glue together deterministic steps (parse a CSV, call a solver, post-process the result). Neither mode alone can express the pattern that the OptiGuide application exhibits, where the LLM must decide *at runtime* whether to invoke a function, and that function's execution must in turn feed back into the LLM's next reply. Hybrid control was not a late addition to the design but a core requirement that drove the interface shape: every built-in reply function returns either a value that is immediately sent back as a message, or a `None` that signals "pass control to the next reply function in the chain". This convention lets a single `generate_reply` call traverse an arbitrary mix of NL, code, and hybrid logic without any special-casing at the interface level.

Dynamic Conversations. Many real applications are not purely two-party, and their topology is not known in advance. My framework provides two mechanisms that together let a developer express multi-party conversations whose structure is decided at run time.

The first mechanism is side conversations inside a custom reply function. Because a reply function has access to the agent's full context and can itself send messages, it is free to pause its current exchange, start a new exchange with a third agent, use the third agent's reply to inform its own, and only then respond to the original sender. The framework does not prescribe which kind of content triggers such side conversations; the decision is expressed as ordinary Python inside the reply function.

The second mechanism is function-call-driven agent dispatch. An LLM-backed agent may emit a function call that, on execution, routes a message to one or more other agents. Because the LLM emits the call at inference time, the topology of downstream interactions becomes a function of what the LLM judges necessary for the current task—not something written into the program statically.

A particular coordinator built on top of these two mechanisms is `GroupChatManager`: it uses a role-play-style prompt to pick which of a set of participating agents should speak on the next turn, then broadcasts that agent's reply to the rest of the group. The same two mechanisms also underpin other dynamic-topology patterns users have built on top of the framework, including nested conversations and selective-listener topologies where only a subset of agents receive a given message.

Comparison with Related Frameworks. Relative to the systems reviewed in Chapter 2, AutoGen's key differentiator is its programmable reply mechanism. Where CAMEL and Multi-Agent Debate [7] rely on fixed conversation patterns, AutoGen enables flexible topologies. Where BabyAGI orchestrates tasks but does not natively integrate code execution, AutoGen treats it as a first-class agent capability. And where MetaGPT prescribes fixed role assignments, AutoGen allows human involvement at arbitrary points and supports dynamic conversation topologies that adapt to the task at hand.

A Minimal Program

To make the abstractions concrete, Listing 3.2 presents the smallest interesting AutoGen program: a two-agent pair that solves a code-driven task. The `AssistantAgent` generates Python; the `UserProxyAgent` executes it in a sandbox and returns the result (or an error) as the next message. The entire control flow—code generation, execution, error recovery, termination—emerges from the auto-reply loop; there is no explicit state machine.

```
from autogen import AssistantAgent, UserProxyAgent
assistant = AssistantAgent(
    name="assistant",
    llm_config={"model":  "gpt-4"})
user_proxy = UserProxyAgent(
    name="user_proxy",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=10,
    code_execution_config={"work_dir":  "coding"})
user_proxy.initiate_chat(
    assistant,
    message="Plot a chart of META and TESLA stock price change
YTD.")
```

Figure 3.2: A minimal two-agent AutoGen program. Five lines of user code suffice to set up a code-writing assistant, a tool-executing user proxy, and kick off the conversation; the auto-reply loop handles iterative code generation, execution, and debugging thereafter.

Two observations follow from this listing. First, the program size is dominated by *con-*

figuration (agent names, LLM settings, human-input policy, code-execution sandbox) rather than by control logic; no conditional branches, loops, or exception handlers appear. Second, substituting `human_input_mode="ALWAYS"` on the `UserProxyAgent` converts this autonomous system into a human-in-the-loop system without any further change—an illustration of the mode flexibility highlighted in § 3.2.

Positioning Against Related Frameworks

Table 3.1 places the conversation-programming paradigm against four contemporary frameworks along five systems dimensions: the central abstraction, who decides which agent speaks next, how control flow is specified, whether humans can intervene mid-conversation, and whether the framework natively supports execution of tool/function calls. The comparison highlights the distinctive combination my framework offers: a *single* agent abstraction that supports *multiple* control-flow modes (natural-language, programmatic, hybrid) and *first-class* human and tool involvement, rather than specialising one axis at the expense of another.

Table 3.1: Positioning of the conversation-programming paradigm against contemporary multi-agent frameworks. Each row is a systems dimension; each column a framework.

Dimension	**AutoGen (mine)**	**LangChain** [4]	**MetaGPT** [11]	**CAMEL** [17]
Central abstraction	Conversable agent unifying LLM, human, tool	Chain of LLM calls plus tool wrappers	Fixed role objects per SOP stage	AI-user and AI-assistant dyad
Speaker selection	NL, code, or hybrid	Chain order, fixed at build	SOP stage order, fixed	Alternating, fixed
Control flow expression	Reply-function chain plus messages	Python code plus chain declaration	Python plus SOP spec	Prompt templates
Human-in-the-loop	First-class, per-agent, three modes	Callback hooks only	No native support	Not supported
Tool or code execution	First-class via user proxy	Core feature, tool-calling API	Core feature, role-scoped	Not first-class

3.4 Applications

I demonstrate the generality of the framework across six application scenarios (Table 3.2), each illustrating a different conversation pattern and domain.

Before walking through the applications, it is worth stating what the table actually measures. Each row represents a different problem domain (mathematics, retrieval, embodied planning, supply chain, coding, games); each row also reports a *key result*, typically a headline accuracy or efficiency gain against the strongest available baseline. Taken together the six applications span a deliberately wide range of conversation patterns—from strictly sequential two-agent exchanges (A1, A2) through nested conversations with tool calls (A3, A4) to four-way group chats (A5) and externally-validated games (A6). The common thread is that each application required no fundamental framework modification: all six were implemented by composing the same `ConversableAgent` abstraction with different reply functions, human-input modes, and speaker-selection strategies. This generality is the strongest single claim I make for the programming model.

A1: Math Problem Solving. On the MATH dataset [10], a two-agent team (`AssistantAgent + UserProxyAgent` with code execution) is evaluated on both 120 level-5 (hardest) problems and the full 5,000-problem test set. On the full dataset,

the system achieves **69.48%** overall accuracy, compared to 55.18% for vanilla GPT-4—a gain of over 14 percentage points. On the level-5 subset, my framework achieves 52.5% accuracy—roughly 1.75× GPT-4's 30.0%—and outperforms every baseline, including ChatGPT with Code Interpreter (48.33%), ChatGPT with Plugins (45.0%), Multi-Agent Debate (26.67%) [18], and LangChain ReAct (23.33%). The key mechanism is that the `UserProxyAgent` executes Python code generated by the assistant, allowing iterative debugging: when execution fails or produces an incorrect result, the assistant generates corrected code automatically through the auto-reply loop. A human-in-the-loop variant further enables solving problems that no automated configuration can handle, such as finding the equation of a plane from geometric constraints.

It is worth unpacking why the math application is such a clear win for the conversation-programming approach. Competition-style mathematics problems are a fundamentally *iterative* domain: an attempt either succeeds on the first pass or fails in an informative way (a traceback, an unreasonable numeric result, a bogus step in the derivation). A single-shot LLM gets one chance to produce the right answer; a two-agent team with a code executor gets as many chances as the auto-reply budget permits, with each failed attempt feeding the next. The improvement from 30.0% to 52.5% on level-5 problems is almost entirely attributable to this debugging loop: the assistant's base ability is unchanged, but it now operates in an environment that rewards iteration. The human- in-the-loop variant pushes further in the same direction, with the human stepping in only at problems where the automated loop has demonstrably stalled.

A2: Retrieval-Augmented Q&A. A retrieval-augmented chat system uses a Chroma [6] vector database with SentenceTransformers [26] embeddings. The critical innovation is *interactive retrieval*: when the retrieved context does not contain sufficient information, the LLM-based assistant replies "UPDATE CONTEXT," triggering additional retrieval attempts. This mechanism achieves a recall of **66.65%** and F1 of 25.88% on Natural Questions [14] (5,332 documents, 6,775 queries), compared to 22.79% F1 / 62.59% recall without interactive retrieval. A separate Dense Passage Retrieval baseline [13], evaluated under a different retriever setup, scores 15.12% F1 / 58.56% recall. The ablation improvement (62.59% → 66.65% recall) isolates the contribution of conversation-driven context refinement. Approximately 19.4% of questions trigger the update mechanism, demonstrating that a significant fraction of queries benefit from conversation-driven context refinement.

A3: ALFWorld. In the ALFWorld embodied decision-making benchmark [28] (134 unseen tasks across six task types), the configuration I use has three agents: a planner (`AssistantAgent`), an actuator (`UserProxyAgent`) that carries the planner's action into the environment, and a third *grounding* agent whose only role is to inject the kind of everyday common-sense that otherwise trips up the planner—constraints such as "an object can only be examined after it has been picked up". The three-agent system achieves **69%** average success (best-of-3: 77%), compared to 54% for the two-agent baseline and 54% for ReAct [32]—a **15 percentage-point gain**. The improvement is particularly large on "Pick Two" tasks (24%→41%), where commonsense grounding about object locations is most critical.

A4: OptiGuide. OptiGuide [16] is a supply-chain optimization assistant that interprets solutions from mathematical programming solvers. My framework reduces its core workflow from over **430 lines to 100 lines** by expressing the three-agent interaction (Commander, Writer, Safeguard) as a conversation pattern rather than explicit control logic. The multi-agent design boosts unsafe-code detection F1 by **8%** with GPT-4 and **35%** with GPT-3.5-turbo compared to a single-agent approach. A user study on a coffee supply-chain scenario shows approximately 3× user time savings compared to ChatGPT with Code Interpreter. Separately, across 2,000 questions on five supply-chain applications, my framework reduces

Table 3.2: Summary of the six application scenarios.

App	#Ag.	Pattern	Domain	Key Result
A1	2–3	Sequential	Math	69.48% (vs. 55.18%)
A2	2	Interactive	Q&A	66.65% recall
A3	2–3	Grounded	Embodied	+15 pp success
A4	3	Guarded	Supply ch.	430→100 LoC
A5	4	Group chat	Coding	11/12 tasks
A6	3	Validated	Games	Legal-move filtering

the number of required user interactions by 3–5×.

The OptiGuide result is worth dwelling on because it is the strongest evidence that conversation programming is *not* just a convenience for researchers but a genuinely productive abstraction for practitioners. The 430-line monolithic OptiGuide implementation entangled three concerns: orchestrating the interaction between the user, the optimisation solver, and the LLM interpreter; enforcing safety checks on generated code; and writing a human-readable explanation of the solver's output. Re-expressing these as three conversable agents—Commander, Writer, Safeguard—separates the concerns cleanly, and each agent becomes a small piece of code with a single responsibility. The 4.3× line-count reduction is not a cosmetic improvement: the decomposed code is easier to test (each agent's reply function is independently testable), easier to extend (a new safety check is a new Safeguard reply function, not a modification to a deeply nested conditional), and easier to reason about (the Commander's control flow is a sequence of messages, not a tangled state machine).

The ALFWorld result (A3) is structurally similar to the math result: the three-agent configuration gives the `AssistantAgent` a second chance to act correctly by consulting a grounding agent, and the grounding agent's role is exactly the kind of common-sense constraint that an LLM struggles to enforce on itself. The 15-percentage-point gain over ReAct comes from the specialisation: rather than having one agent that plans, executes, and self-critiques, the three-agent team divides these responsibilities, and each agent's prompt can be optimised for its specific role.

A5: Dynamic Group Chat. Four agents share a single chat—a user proxy, a coding engineer, a critic, and a code executor—and I evaluate the configuration on twelve hand-written complex coding tasks. Who speaks next is chosen by the `GroupChatManager` via a role-play prompt that describes the agents' personas and asks the LLM to pick the most-appropriate next speaker. Under that selection policy the team clears eleven of twelve tasks on GPT-4, up from nine out of twelve on a two-agent baseline, while using only 4.5 LLM calls on average (vs. 6.8) and experiencing zero termination failures (vs. 3). A task-based speaker selection variant achieves only 8/12 successes, confirming that the role-play approach better captures the nuances of turn-taking in collaborative coding.

A6: Conversational Chess. The chess application pits two player agents against each other and mediates their moves through a third, board-keeping agent. The board agent's job is purely mechanical: translate whatever natural-language move the player has uttered into UCI notation, check it against the live board, and either apply it or reject it. On a rejection the board agent asks the player to re-propose—a pattern made trivial by the `register_reply()` mechanism. An ablation removing the board agent and relying solely on a prompt ("you should make sure both you and the opponent are making legal moves") results in frequent illegal moves and game disruptions, demonstrating that tool-backed grounding is essential for reliable game play.

3.5 Discussion

My framework's contribution to the systems stack is a *unified programming model* that treats multi-agent conversation as the first-class programming primitive. Its strengths include:

- A single `ConversableAgent` abstraction that subsumes LLMs, humans, and tools under a uniform interface;
- Flexible conversation topologies (sequential, nested, group) that adapt to diverse application requirements;
- Native human-in-the-loop support at arbitrary points, allowing seamless transitions between automated and interactive modes;
- A modular `register_reply()` mechanism that separates agent logic from conversation orchestration, reducing boilerplate by up to 4.3× (as demonstrated in the OptiGuide application).

From a systems perspective, AutoGen can be understood as a *message-passing programming model* in which agents are actors and conversations are channels. The auto-reply event loop is analogous to an event-driven runtime: each `generate_reply()` invocation is a callback triggered by a message arrival. This design choice makes the framework highly expressive—in principle, a wide range of conversation topologies can be constructed by composing reply functions—but it also means that the framework provides no explicit notion of task dependencies, parallel execution, or resource allocation.

These limitations manifest concretely when I scale my framework to complex workloads, and they divide naturally into three classes: the absence of a principled evaluation framework, the absence of a task-graph representation, and the absence of parallel execution.

First, my framework measures success through *task-specific accuracy*—MATH scores, ALFWorld success rates, 8% F1 gains for OptiGuide—but provides no general-purpose metrics for output quality, computational efficiency, or information density. Real-world deployment demands exactly the kind of multi-dimensional evaluation that a single numeric accuracy figure cannot capture: a poster pipeline that scores 116 on a quiz probe at $0.005 per poster is a different kind of system from one that scores 112 at $1.87, yet both might earn the same "success rate" under a narrow quality metric. Chapter 4 addresses this gap directly by proposing a four-dimensional evaluation framework—visual, textual, holistic, and quiz-based—that treats generation quality, cost, and information transfer as distinct axes of evaluation.

Second, in the design presented here, conversations proceed sequentially; there is no mechanism to express that subtasks are *independent* and could execute concurrently.[1] A four-panel poster whose Painter–Commenter loops are structurally independent, for example, is squeezed through the auto-reply chain one reply at a time, leaving a potential 69.3% speed-up (the parallelization figure I report in Chapter 4) on the table. The underlying issue is that the reply-function chain is *Turing-complete but representation-opaque*: a runtime that inspected an AutoGen program at a semantic level could in principle recover the implicit DAG, but no such interface is provided today.

Third, even within a single conversation, all `generate_reply()` calls are synchronous; agents in a group chat are invoked one at a time by the `GroupChatManager`. This is a deliberate design choice—synchronous turn-taking makes reply functions composable and debuggable—but it is also a hard ceiling on throughput. Chapter 5 will show that lifting

[1] The design I present here corresponds to the original framework; the graph-based workflows and async/distributed runtimes in later iterations are out of scope for this thesis and do not bear on the argument.

this ceiling requires introducing a runtime layer that schedules task vertices rather than conversation turns, and manages context through semantic routing rather than sequential broadcast.

A fourth, more subtle limitation deserves separate mention: the conversation-programming model implicitly assumes that *all inter-agent coordination can be expressed as natural-language message passing*. This works well for deliberation-heavy tasks (debate, code review, step-by-step planning) where the information to be communicated is fundamentally linguistic. It becomes awkward for tight data dependencies—e.g., passing a structured asset library between parser and planner stages, or handing off a tensor between model-inference agents—where a typed interface would be more natural and less error-prone. The workaround in my framework—serialising structured data through JSON embedded in natural-language messages—imposes a hidden tax on every such exchange, both in tokens (the JSON payload is re-tokenised with every message) and in reliability (model errors in parsing or emitting JSON become a new class of bug with no compile-time signal).

The first three limitations motivate, respectively, the workload layer (Chapter 4), the runtime layer (Chapter 5), and Chapter 6's cross-layer synthesis. The fourth is a cleaner piece of future work that would require a typed message protocol orthogonal to the natural-language channel, and I return to it in § 7.2.

Revisiting the Strengths Against the Limitations

The strengths listed at the top of this section are not accidents; they are direct consequences of the design choices behind the conversation-programming paradigm. The single `ConversableAgent` abstraction is a strength because it lets a developer reason about "an agent" without worrying whether that agent will be backed by an LLM, a human, or a tool; but it is also the root cause of the typed-interface limitation discussed above, because the abstraction's message channel is untyped natural language. Flexible conversation topologies are a strength because they let applications choose the pattern that suits the task; they are also the root cause of the runtime scheduler's difficulty, because a topology defined only at run time is harder to schedule than one declared statically. Native human-in-the-loop is a strength because it makes safety reviews and oversight first-class; it is also a source of latency, because a human consultation is synchronous and slow compared to an LLM call.

The framework thus sits at a particular point on a design spectrum: extremely expressive, at the cost of implicit task structure and synchronous control flow. The other two layers of this thesis—the workload-and-evaluation layer and the runtime layer—take this design as a given and compensate for its limitations where needed. Chapter 4 provides the multi-dimensional evaluation that the programming model alone cannot offer; Chapter 5 provides the task-graph representation and parallel execution that the reply-function-chain primitive alone cannot express. Seen from the stack's perspective, each layer's limitations are the next-layer's motivation, and the three layers together cover ground none of them could cover alone.

3.6 Summary

In this chapter I introduced the conversation-programming paradigm: a unified `send/receive/generate_reply` interface on a common `ConversableAgent` base class, with computation and control flow expressed through inter-agent messages. Section 3.4 showed that this abstraction carries real-world applications from mathematics and coding through retrieval-augmented generation and group decision making. The resulting design places few demands on the agent implementation while admitting flexible,

programmable orchestration—properties I exploit in Chapter 5, where a runtime schedules conversable agents at scale.

Chapter 4

Workload and Evaluation: Multi-Modal Poster Generation

4.1 Motivation

Paper2Poster addresses a question that the programming-model layer of Chapter 3 leaves open: *What does a realistic multi-agent workload look like, and how do we measure success?* It frames scientific-poster generation as a canonical multi-modal, long-context, iterative-refinement task and contributes both a curated benchmark and a multi-dimensional evaluation framework.

From the systems perspective, the workload-and-evaluation layer plays a dual role. First, it defines a *workload specification*: the computational demands, input/output characteristics, and decomposition structure of a representative multi-agent task. This is analogous to benchmark suites like SPEC or MLPerf that characterize system performance across diverse workloads. Second, it provides an *evaluation framework*: a set of metrics that capture multiple dimensions of quality, enabling principled comparison of different agent configurations, programming abstractions, and runtime strategies. Without such a layer, improvements at the programming or runtime level cannot be meaningfully assessed.

4.2 Task Definition and Benchmark

The task is to transform a full scientific paper into a single-page conference poster. This requires extreme compression: on average, 12,155 words across 22.6 pages with approximately 20,370 tokens and 22.59 figures must be distilled into 774 words (1,416 tokens) on one page with 8.7 figures—a **14.4**$\times$ token compression ratio (equivalently, 15.7$\times$ in word count and 22.6$\times$ in page count) and a 2.6$\times$ figure reduction ratio (Figure 4.1). Pages undergo the most extreme compression of any dimension, while figures are reduced only modestly, reflecting the importance of visual content on a poster.

Why this is a canonical MAS workload. Poster generation stresses the full spectrum of capabilities that a multi-agent system must bring to any realistic document-generation task, and does so in a form that makes each stress testable and measurable. I argue below that five properties—long-context reasoning, multi-modal processing, iterative refinement, decomposable structure, and measurable quality—together make the task an ideal benchmark for the programming-model and runtime layers alike.

The *long-context* challenge is blunt but unavoidable. Input papers span tens of thousands of tokens (averaging 20,370), and the agents must maintain coherence across a large document—tracking which claims appear where, which figures support which results, and which sections carry the argument's load. A system that silently drops a key ablation from page 18 has produced a poster whose information score will collapse under the PaperQuiz probe described below, even if the visual layout looks polished.

Multi-modal processing goes beyond simply *seeing* the figures: agents must jointly reason about text, figures, tables, and layout. A correctly placed figure captions itself by adjacency to the right paragraph; a misplaced one becomes a decorative element. The Painter–Commenter loop I introduce later in this chapter treats this joint reasoning as a first-class requirement rather than an after-the-fact rendering step, and the four evaluation dimensions

below reward it as such.

Iterative refinement is the natural rhythm of high-quality poster production. Draft, critique, revise: a one-shot generator that must commit to a layout before rendering any panel will be dominated by a system that can re-render a panel after observing text overflow. This iterative rhythm maps directly onto the multi-agent feedback loop—a Commenter agent critiquing a Painter's output—and is precisely the class of workload where AutoGen-style conversation programming shines.

Decomposable structure is the property that turns the benchmark from a monolithic quality test into a stress test for parallelism. The overall task naturally decomposes into parsing, planning, and rendering stages; each stage can be assigned to a specialized agent; and each stage itself further decomposes—a poster with eight panels yields eight structurally independent Painter–Commenter subtasks at the rendering stage. Chapter 5's runtime layer will exploit exactly this structure to cut wall-clock time by 40–70%.

Finally, *measurable quality*: unlike open-ended generation tasks ("write a story about X"), scientific posters admit concrete ground-truth references and multiple evaluable dimensions. Visual similarity, textual coherence, holistic assessment, and information transfer are all separately quantifiable, and they do not reduce to a single axis—the 4o-Image baseline I report later achieves the highest visual similarity but the worst perplexity, producing posters that are visually attractive but textually noisy. A benchmark that averaged these dimensions into a single number would obscure exactly the kind of trade-off that matters for system design.

Positioning against related document-generation tasks. Before describing the benchmark data, it is worth saying briefly what Paper2Poster is *not*. It is not a summarisation benchmark: reducing a paper to a paragraph is a strictly easier task than reducing it to a structured, visually laid-out poster that must preserve figures and bullet-pointed content. It is not a layout-only benchmark either: a system that produces a pixel-perfect empty grid fails as badly as one that produces a cluttered wall of text. And it is not a generic image-generation benchmark—the "image" being produced is structured, textual, and evaluable at the panel level, unlike a free-form illustration. The closest analogues in the literature are POSTERSUM [27] (the data source, which frames the task as summarisation) and PPTAgent [33] (slide generation, which shares the panel-level discretisation but targets a much more permissive canvas); neither subsumes the combined multi-modal-plus-layout-plus-information-transfer challenge.

Benchmark Data. The benchmark comprises **100 paper–poster pairs** drawn from the test split of the POSTERSUM dataset [27], covering NeurIPS (35), ICML (37), and ICLR (28) from 2022–2024, stratified by year (33 from 2022, 33 from 2023, 34 from 2024) for temporal balance. Papers range from 15 to 50 pages (including supplementary material) and span 280 distinct topics across computer vision (19%), NLP (17%), reinforcement learning (10%), and others. The test split is specifically chosen to minimize training-data overlap with the LLMs used in evaluation.

4.3 Four-Dimensional Evaluation

I introduce a four-dimensional evaluation framework (Table 4.1) that goes well beyond single-number accuracy. The design principle behind the framework is straightforward: each dimension targets a distinct aspect of poster quality; each dimension is independently computable without requiring a separate human study; and each dimension yields a quantitative scale that allows direct cross-method comparison.

Before elaborating each dimension, it is worth justifying why four dimensions is the right count. The most obvious simpler alternative would be a single holistic score—"on a scale of

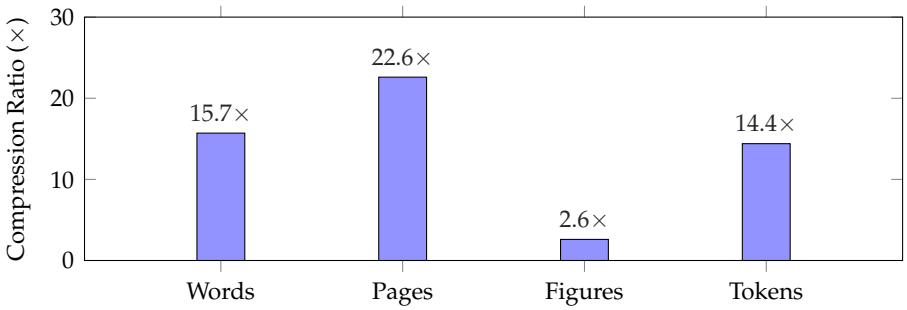

Figure 4.1: Paper-to-poster compression ratios across four dimensions. An average paper (12,155 words, 22.6 pages, 22.59 figures, 20,370 tokens) is compressed into a single poster (774 words, 1 page, 8.7 figures, 1,416 tokens). Pages undergo the most extreme compression (22.6×), while figures are reduced by only 2.6×, reflecting the importance of visual content in posters.

1 to 5, how good is this poster?"—and this has a serious defect: a judge that sees a poster with a beautifully balanced layout, crisp figures, and subtly wrong content (a claim misstated from the source paper) will score it highly on aesthetic criteria, moderately on clarity, and has no principled way to penalise the fact that the information is wrong. The holistic judgement ends up dominated by whatever criterion the judge finds most salient, which in practice is often visual rather than informational. A richer three-dimensional alternative (visual, textual, holistic) improves on the single score by exposing the aesthetic/linguistic trade-off, but still cannot distinguish a text-heavy poster that covers everything at low clarity from a concise poster that covers the main claims clearly: both score similarly on textual coherence (perplexity is insensitive to coverage) and on VLM-as-Judge content completeness (the judge rewards exhaustiveness). The missing axis is information transfer to a downstream reader—which is precisely what the fourth dimension, PaperQuiz, adds.

(i) Visual Quality. Two CLIP-based [25] metrics capture visual fidelity. *Visual similarity* computes the cosine similarity between AltCLIP [5] image embeddings of the generated and ground-truth posters:

$$s_{\text{VS}} = \cos\big(\mathbf{z}_I(\hat{P}),\ \mathbf{z}_I(P^*)\big). \tag{4.1}$$

Figure relevance measures how well each of the N_f figure crops f_i in the poster aligns with its corresponding section text t_i:

$$s_{\text{FR}} = \frac{1}{N_f}\sum_{i=1}^{N_f}\cos\big(\mathbf{z}_I(f_i),\ \mathbf{z}_T(t_i)\big). \tag{4.2}$$

(ii) Textual Coherence. Standard perplexity (PPL) under Llama-2-7B [30] quantifies the fluency and local coherence of poster text:

$$\text{PPL} = \exp\Big(-\frac{1}{n}\sum_{i=1}^{n}\log p(w_i \mid w_{<i})\Big). \tag{4.3}$$

(iii) Holistic Assessment (VLM-as-Judge). A Vision-Language Model (VLM)—specifically GPT-4o—is prompted as an automated judge, outputting structured JSON with a justification and a score from 1 to 5 for each of six criteria grouped into two dimensions:

Aesthetic Score (averaged over three criteria):

1. **Element Quality**: how crisp, well-rendered, and stylistically uniform the graphic elements are;

2. **Layout Balance**: how well text and figures are laid out, sized relative to each other, and spaced across the page;

3. **Engagement**: the extent to which the poster's design holds a viewer's attention on a first look.

Information Score (averaged over three criteria):

4. **Clarity**: how readable the poster's sentences are at a granular level—grammar, phrasing, and word choice;

5. **Content Completeness**: whether the poster covers all the paper's key sections at a useful level of detail;

6. **Logical Flow**: whether the poster's sections follow one another in an order that a reader can easily track.

The overall VLM-as-Judge score is the average of Aesthetic and Information scores. Five independent evaluation runs on PosterAgent-4o yield standard deviations below 0.024 across all metrics, with an overall average standard deviation of just 0.005, confirming strong reproducibility.

(iv) PaperQuiz. The most novel evaluation metric addresses a fundamental question: *How much of a paper's content can a reader extract from the poster alone?* The protocol works as follows. First, each paper's PDF is converted to Markdown and submitted to OpenAI's o3 model as an examiner to generate **100 multiple-choice questions**:

- **50 verbatim questions** are directly answerable from the text, spanning 13 content aspects (e.g., main contribution, dataset description, experimental setup, baseline comparisons, ablation results).
- **50 interpretive questions** require higher-level comprehension beyond verbatim text, spanning 10 conceptual dimensions (e.g., methodology rationale, limitations, broader impact, connections to related work).

Six VLM readers—three open-source (LLaVA-OneVision-Qwen2-7B-ov-hf, Phi-4-multimodal-instruct, Llama-4-Scout-17B-16E-Instruct) and three closed-source (o3, GPT-4o mini, Gemini 2.0 Flash)—answer questions *solely from the poster image*, with no external knowledge allowed. Each answer must cite the poster region supporting it; if the information is not found, the reader responds "NA."

A concrete example makes the protocol easier to trust. For a paper on sparse attention, the o3 examiner might produce a verbatim question such as "What is the attention sparsity level reported in the main experiment? (A) 50% (B) 75% (C) 90% (D) 95%" (answer: C, read directly from a results table in the paper), alongside an interpretive question such as "Which of the following best explains why the sparsity pattern is learned rather than fixed? (A) it allows the network to adapt to input length; (B) it reduces memory usage; (C) it improves interpretability; (D) it is required for the convergence proof" (answer: A, inferable from the methodology section but not written as a sentence). A VLM reader receives only the generated poster image plus the question stem; it must recover (C) or (A) from whatever the poster surfaces, or respond "NA" if the information is not present.

The question-generation prompt explicitly asks the examiner to draw 50 verbatim questions from 13 content aspects (main contribution, dataset description, experimental setup, baseline comparisons, ablation results, and so on) and 50 interpretive questions from 10 conceptual dimensions (methodology rationale, limitations, broader impact, connections to related work, etc.), yielding broad coverage of both "what does the paper say?" and "what

Table 4.1: Paper2Poster evaluation dimensions.

Dimension	Metric(s)	What It Captures
Visual Quality	CLIP similarity, figure relevance	Visual fidelity and figure–text alignment
Textual Coherence	Perplexity (PPL)	Fluency and local coherence of text
Holistic (VLM)	6 criteria, 1–5 scale	Aesthetic appeal and information quality
PaperQuiz	100 MCQs, density-augmented	Information density and comprehensibility

does the paper mean?" The two-way split is consequential for system comparison: a poster that reproduces result tables verbatim can score highly on verbatim questions while failing the interpretive ones; a poster that distils the paper's argument into a single clear takeaway can do the reverse. The total PaperQuiz score weights both classes equally, so a system must do well on both to score highly.

A *density-augmented* scoring formula adjusts the raw score s_r based on poster length, reducing the density bonus for verbose posters that achieve high quiz scores simply by including more text:

$$s_a = s_r \cdot \Big(1 + \frac{1}{\max(1,\ L/W)}\Big), \tag{4.4}$$

where $s_r \in [0, 100]$ is the raw quiz score, L is the poster's text length, and W is the median length of human-designed posters. This formula has three desirable properties: (1) overly long posters ($L \gg W$) receive $s_a \approx s_r$, losing the density bonus; (2) the score never drops below s_r—as $L \to \infty$, s_a approaches s_r from above; (3) extreme brevity is not rewarded beyond a cap of $s_a = 2s_r$ (when $L \leq W$).

4.4 PosterAgent Pipeline

The best-performing system is **PosterAgent**, itself a multi-agent pipeline comprising three stages (Figure 4.2).

Parser (Global Organization). The Parser stage accepts a PDF as input and yields what I call an *asset library*: a pair of dictionaries, one indexing textual content and one indexing visual content. The textual dictionary maps each section heading in the paper to a short summary of that section's prose, preserving the paper's logical organisation (Introduction, Method, Results, and so on) while compressing each section's body into something much shorter than the original. The visual dictionary maps each figure or table's caption to the corresponding extracted image file, so that downstream stages can look up a figure by its caption without re-parsing the PDF. Text and visuals are extracted in a single pass.

Two Markdown converters, Marker [24] and Docling [19], are run side by side on each page because each has different failure modes on atypical document layouts; the outputs are combined page-by-page before an LLM pass produces the structured dictionary representation. The Parser is a critical stage: the 14.4× token compression achieved by the whole pipeline is effectively fixed here, and any content that does not survive the Parser is gone for the rest of the pipeline.

Planner (Local Organization). With the asset library in hand, the Planner is responsible for three linked decisions. First, it must *match* each visual asset to the paper section where it is most appropriate (the teaser image to the introduction, the ablation table to the experiments,

and so on); this is an LLM-driven semantic alignment that outputs (section, visual) pairs. Second, it must produce a *layout*: a deterministic recursive algorithm estimates the content length of each section (word count for text, size estimate for visuals), then partitions the poster rectangle via binary splits until each leaf of the split tree holds exactly one panel. The recursion keeps reading order (left-to-right, top-to-bottom) and aspect-ratio preservation as invariants, so every split is well-defined. Third, it must *iterate* over each section, condensing its text into tiered bullet points at a level of compression suited to a poster panel—this is the representation the Painter stage ultimately renders.

Painter–Commenter (Local Refinement). The rendering stage is a loop, not a feed-forward pipeline, and that is the most important structural choice in the design. On each iteration, the **Painter** does two things: an LLM compresses the section's bullets into a tight panel-ready form, and a deterministic helper emits Python code that calls into `python-pptx` [3] to paint the panel into a PowerPoint slide. The code is then run, producing a rendered panel image.

The **Commenter** inspects the rendered panel through a VLM. Rather than consider the whole poster, the Commenter zooms into the panel region alone—local context, local judgement. To make the judgement reproducible, the Commenter's prompt carries two calibration images: one deliberately broken (text overflowing), one deliberately balanced. Against those anchors, the Commenter classifies the current panel into one of three states: text overflow, excessive whitespace, or acceptable. The first two states trigger another round—the Painter tightens font size, line spacing, or truncates content and re-emits—and the loop ends either when the Commenter accepts the panel or when an iteration cap is reached, so the compute budget is bounded even under worst-case input.

Variants. I evaluate two PosterAgent configurations: **PosterAgent-4o** uses GPT-4o for both the internal LLM and VLM Commenter, while **PosterAgent-Qwen** is fully open-source, using Qwen-2.5-7B for text generation and Qwen-2.5-VL-7B [2] for the Commenter. The existence of a competitive open-source variant suggests that, in this setting, the pipeline architecture contributes more to output quality than model scale alone.

Design choices worth highlighting. Three design decisions are worth calling out explicitly, because each of them turned out to matter more than I expected when comparing against baselines. The first is the *asset library* abstraction. Rather than passing the full PDF (or a sliding window over it) to every downstream stage, the Parser produces a compact structured representation once—a dictionary mapping section headings to summaries, and figure captions to image paths. All subsequent stages operate on this compact representation. This is the single biggest contributor to PosterAgent's token-efficiency advantage over OWL-4o: the Planner never re-reads the source paper, even though it needs to make dozens of layout decisions that semantically depend on the paper's content. The asset library acts as a compiled representation of the document that makes those decisions cheap.

The second is the *binary-tree layout*. It would have been tempting to let the Planner produce a free-form layout—a list of (x, y, width, height) tuples, say, one per panel—and rely on an LLM to keep the layout sensible. In practice, binary-tree partitioning imposes enough structure that the LLM does not have to reason about global constraints like panel non-overlap, reading order, or aspect-ratio preservation: the recursive partitioning algorithm guarantees these by construction, and the LLM only has to decide the split direction and split ratio at each level. The division of labour between deterministic algorithm and LLM creativity is essential here—PPTAgent's failure mode (sparse panels, repeated content) is arguably a symptom of giving too much layout responsibility to the LLM.

The third is the *in-context reference* trick in the Commenter. A Commenter that merely asks "does this panel look right?" produces inconsistent judgements across runs. A Commenter that sees two calibration examples—one deliberately overflowing, one balanced—

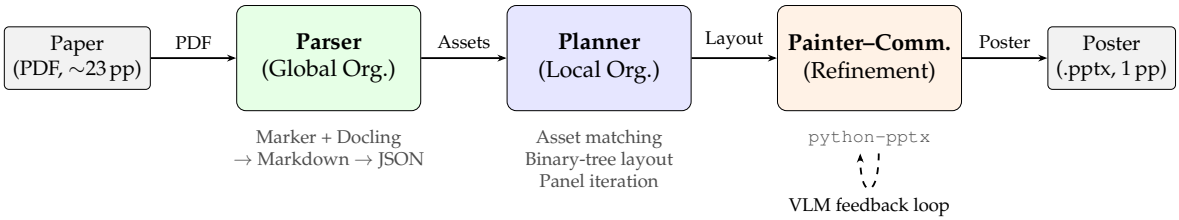

Figure 4.2: PosterAgent pipeline. The Parser builds an asset library from the paper PDF; the Planner organizes assets into a binary-tree layout; the Painter–Commenter iteratively renders and refines each panel via VLM feedback.

alongside the panel under review produces stable verdicts. The technique is cheap (adds two images to the prompt) and contributes much of PosterAgent-Qwen's surprising competitiveness: a 7B VLM can make reliable layout-quality judgements when given concrete anchors, even though the same VLM on the same task without anchors is unreliable.

4.5 Results

Table 4.2 compares PosterAgent against baselines and human posters across all evaluation dimensions.

Quality. PosterAgent-4o achieves a VLM-Judge overall score of **3.72**, approaching the human ground-truth score of 3.77 and substantially outperforming all baselines: 4o-HTML (3.52), OWL-4o [12] (3.19), and PPTAgent-4o [33] (2.11). On PaperQuiz (density-augmented, Eq. (4.4)), PosterAgent-4o scores **116.13**, surpassing not only all baselines but also the human ground-truth posters (111.78) *on this specific metric*. This suggests that PosterAgent's structured compression produces posters from which VLM readers can extract information effectively relative to poster length, though it does not imply overall superiority over human-designed posters (which score higher on Element Quality and Layout Balance).

Efficiency. PosterAgent achieves strong token efficiency, using only 101.10K tokens (GPT-4o variant) and 47.55K tokens (Qwen variant)—reducing token usage by 72–87% compared to OWL-4o (361.10K tokens). In monetary terms, PosterAgent-4o costs $0.55 per poster (70% reduction from OWL-4o's $1.87), while the fully open-source PosterAgent-Qwen costs only **$0.005 per poster**—over 374× cheaper than OWL-4o. Parallelizing panel-level content generation reduces generation time from 176.69s to 54.16s (69.3% reduction), bringing total runtime from 281.48s to 166.80s (40.7% improvement).

Key Findings. Four insights deserve highlighting.

The first is that *engagement*, in the VLM-as-Judge sense of drawing and holding a viewer's attention, is uniformly the hardest axis: every automated method scores below 3, and even human posters land only at 2.70. Whatever creative-visual-composition skill produces genuinely engaging posters is not something current pipelines—mine included—reliably capture. Engagement is the open problem the evaluation framework most clearly surfaces.

The second is that strong visual similarity can actively mislead. The 4o-Image baseline scores the highest visual-similarity (0.76) among all methods but also the worst perplexity (77.13), and its VLM-as-Judge score sits near the bottom of the list. A poster that superficially resembles the ground truth can still convey mangled content, and a single-score evaluator would miss the collapse entirely.

The third is that structural compression beats raw model scale on information-transfer metrics. PosterAgent-Qwen uses a 7B-parameter VLM and still outperforms much larger

Table 4.2: Poster generation methods comparison.

Method	VLM-Judge (Overall)	PaperQuiz (s_a)	Tokens (K)	Cost ($)
GT Poster (human)	3.77	111.78	—	—
Paper (oracle)	3.90	71.52	—	—
4o-HTML	3.52	108.13	20.67	0.14
4o-Image[1]	2.33	100.36	—	—
OWL-4o	3.19	100.80	361.10	1.87
PPTAgent-4o	2.11	62.49	255.73	1.79
PosterAgent-4o	**3.72**	**116.13**	101.10	0.55
PosterAgent-Qwen	3.66	114.65	47.55	0.005

[1]4o-Image is a one-shot image-generation baseline with no separately measurable token or USD cost; it is therefore omitted from the Pareto analysis in Figure 4.3.

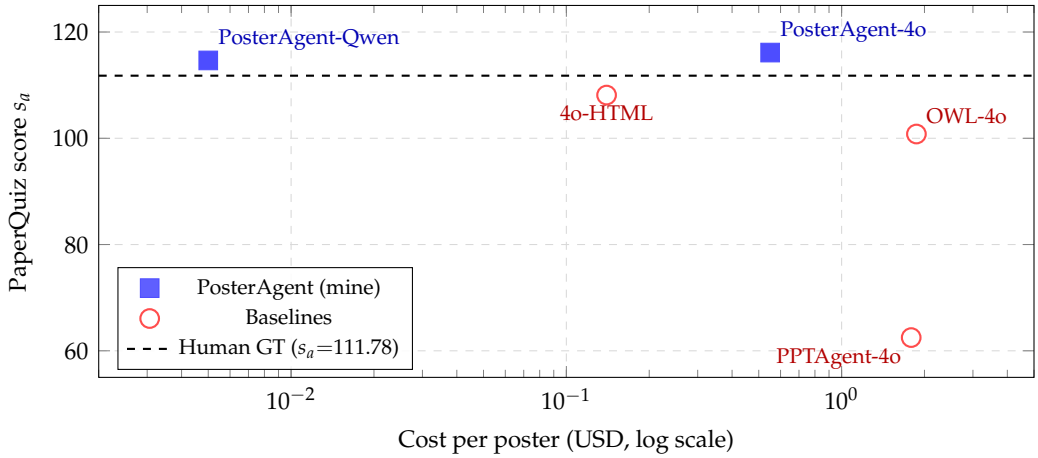

Figure 4.3: PaperQuiz score versus cost per poster on a log-scale cost axis. PosterAgent-Qwen and PosterAgent-4o occupy the upper Pareto frontier: both surpass every baseline on information transfer, and PosterAgent-Qwen is roughly 28× cheaper than the next-cheapest baseline (4o-HTML) while achieving a higher score. The dashed line marks the human ground-truth poster score.

baselines on the density-augmented PaperQuiz, precisely because the structured panel layout gives the VLM reader a scaffold it can parse reliably—the layout does cognitive work that the reading model would otherwise have to do from scratch.

The fourth is related but distinct: the *stronger* VLM readers benefit more from well-structured posters than weaker readers do. The effect is consistent with the intuition that a better reader is better at extracting information from a structure the designer has put effort into, and it suggests that layout investment pays compounding returns as reader capability improves.

Figure 4.3 places these numbers on a quality–cost plane and makes the Pareto structure explicit: PosterAgent-Qwen dominates every baseline on both axes simultaneously, and PosterAgent-4o pushes the quality frontier slightly beyond the human ground truth while remaining 3–4× cheaper than OWL-4o or PPTAgent-4o.

Per-Baseline Failure Analysis

A clearer picture of why PosterAgent wins emerges from analysing where each baseline fails. 4o-Image is the most interesting failure mode: the system produces an image-generation output that looks glossy (visual similarity 0.76, the highest any method achieves)

but carries the lowest textual coherence of any system, with a perplexity of 77.13—more than an order of magnitude above the next-worst baseline. The failure is not of the backbone model; it is of the pipeline design. A single image-generation call is structurally incapable of iterating on local panels after producing the full poster, so text-overflow or typographic errors generated at one point cannot be repaired at another. The Painter–Commenter loop in my pipeline is exactly the missing piece.

OWL-4o fails at the opposite axis: its output quality is competitive (VLM-Judge 3.19, PaperQuiz 100.80), but it pays 361.1K tokens and $1.87 per poster to achieve it. The pipeline is essentially an autonomous planner that explores the full document at each stage, trading token budget for convergence. My Parser–Planner–Painter decomposition achieves comparable quality at roughly a third of the token budget because the Parser pre-compresses the document into an asset library once, and the Planner then operates on the compressed representation for every downstream decision.

PPTAgent is the clearest quality failure: VLM-Judge 2.11 and PaperQuiz 62.49, both the worst in the comparison. The cause is generality mismatch. PPTAgent is designed for slide generation, where the canvas is much more permissive (many slides, modest information density per slide) than a poster (one page, high density). When repurposed to poster generation, its layout heuristics produce sparse panels and repeated content; its tables are often truncated; and the VLM judge's Content Completeness dimension drops accordingly.

4o-HTML is the most competitive baseline: it scores 3.52 VLM-Judge and 108.13 PaperQuiz at only $0.14 per poster, within striking distance of PosterAgent-4o on quality and cheaper per unit. The gap, small but persistent, comes from the in-context reference prompt my Commenter uses: the calibration examples—one deliberately over-stuffed, one balanced—teach the critic to flag text overflow more reliably than a same-model baseline operating without them.

The workload-and-evaluation layer's contribution is thus twofold: it supplies a concrete, demanding workload that reveals the limitations of naïve agent pipelines, and it provides a reusable evaluation methodology. The parallelization gain reported above hints at the potential for a dedicated runtime layer—one that could identify and exploit such parallelism *automatically*, without manual engineering.

Two limitations deserve note. First, the benchmark is restricted to AI conference posters (NeurIPS, ICML, ICLR); it remains unclear whether the evaluation metrics—particularly PaperQuiz—transfer to other document genres (e.g., medical or humanities posters) where visual conventions differ substantially. Second, although panel-level subtasks are structurally independent—and I demonstrate a 69.3% time reduction by parallelizing panel generation—the default PosterAgent implementation processes the Painter–Commenter refinement loop for each panel *serially*. This means that realizing the full parallelism potential requires explicit engineering effort, motivating the kind of automatic scheduling that a dedicated runtime (Chapter 5) could provide.

What the Four Dimensions Measure Together

Having reported the individual dimensions and baseline numbers, it is worth pausing on what the four dimensions measure *jointly* that none of them measures alone. Visual similarity catches aesthetic drift from the reference poster; perplexity catches textual nonsense. VLM-as-Judge catches layout balance, engagement, and content completeness, with standard deviations small enough to be meaningfully compared across runs. PaperQuiz catches something the other three cannot: whether the information in the source paper survived the compression. A poster could score well on the first three dimensions while scoring poorly on the fourth (a visually striking poster that mis-states the paper's main result), and vice versa (a plain poster whose content is accurate but whose layout is unremarkable).

Conversely, the four dimensions are *not independent*. Textual coherence correlates with VLM-as-Judge clarity; visual similarity correlates weakly with holistic aesthetic; the density-augmented PaperQuiz score correlates with both VLM content completeness and textual coherence. The design rationale for keeping them separate rather than collapsing into a weighted sum is that the trade-offs matter: a system designer benefits from seeing that a pipeline change improves VLM-as-Judge by 0.3 points while reducing PaperQuiz by 5 points, so they can decide whether the trade-off is worth it for their use case. A single scalar would hide exactly this kind of signal.

A related methodological choice: I evaluate with both closed-source and open-source VLM readers on PaperQuiz rather than picking one. A single reader would let me report a single number but would also let a pipeline hidden-tune to that reader's preferences. Six readers with a reported mean and standard deviation forces the pipeline to produce posters that are legible to a *diversity* of VLMs, not just to GPT-4o—a property I consider load-bearing for real-world deployment, where a poster's audience is diverse and mostly not the system that generated it.

4.6 Summary

In this chapter I introduced Paper2Poster, a workload that stresses the full spectrum of multi-agent capabilities—multimodal grounding, long-context reasoning, and quality-critical layout—together with a four-dimensional evaluation that measures visual quality, textual quality, holistic judgment, and learned information transfer via a quiz probe. The PosterAgent pipeline demonstrates that competitive quality is achievable with strong open-source base models at modest cost. The per-baseline failure analysis clarified which design choices matter most: the asset-library compression that turns the document into a reusable compact representation; the binary-tree layout algorithm that offloads global constraints from the LLM; and the in-context reference trick that makes the Commenter's layout judgements reliable even with a 7B-parameter VLM. The workload and its evaluation serve throughout the remainder of the thesis as a concrete lens for cost-aware system design.

Chapter 5

Runtime: Dynamic Task-Graph Scheduling

5.1 Motivation

While the programming model of Chapter 3 defines *how* agents interact and the workload layer of Chapter 4 defines *what* they must accomplish, a critical question remains: *How do we execute multi-agent workflows efficiently?* I answer this with DynTaskMAS, a dynamic task-graph–driven runtime for asynchronous, parallel LLM-based multi-agent systems.

5.2 Architecture Overview

Traditional multi-agent systems execute tasks serially, wasting computational resources when independent subtasks could run in parallel. Even frameworks like AutoGen, which support flexible conversation topologies, process each `generate_reply()` call synchronously. My runtime addresses this fundamental limitation with four tightly integrated components (Figure 5.1):

1. **Dynamic Task Graph Generator (DTGG)**: the component responsible for turning a user-supplied task description into a structured DAG that names subtasks and their precedence. The DTGG decomposes tasks hierarchically down to a configured granularity, and—crucially for an LLM-driven setting—re-edits the graph at runtime whenever a subtask discovers new work or a dependency turns out to be finer-grained than expected.

2. **Asynchronous Parallel Execution Engine (APEE)**: the component that actually runs ready vertices across a pool of LLM-backed agents. The APEE is itself factored into a handful of sub-modules—a task scheduler, an execution queue manager, an agent pool manager, a load balancer, and an asynchronous communication handler—each owning one axis of the dispatch problem.

3. **Semantic-Aware Context Management System (SACMS)**: the component that moves information between agents. Rather than broadcast every update, the SACMS maintains a hierarchical context store and uses semantic tags and embeddings to decide which agents genuinely need each update, striking a balance between stale information and communication overhead.

4. **Adaptive Workflow Manager (AWM)**: the component that observes the runtime and adjusts it. The AWM watches performance metrics $M(t)$—queue depths, per-agent utilisation, observed vs. predicted execution times—and nudges configurations or resource allocations in response. Unlike the other three, it does not own an execution path; it is an out-of-band controller.

The four components form a layered architecture with bidirectional information flow: the DTGG feeds task graphs to the APEE and SACMS; the APEE and SACMS exchange context and scheduling information; and the AWM monitors all components, adjusting configurations and resource allocations in response to runtime metrics.

It is worth elaborating on the *separation of concerns* this architecture enforces. The DTGG's sole job is to produce a well-formed task graph; it does not care how the graph is executed.

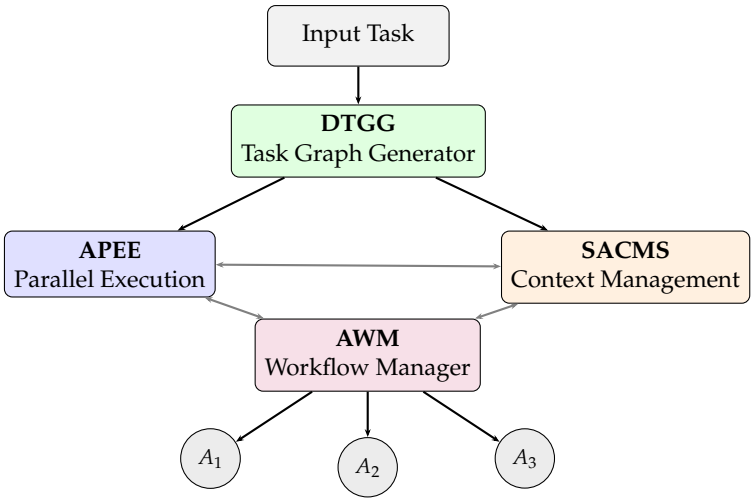

Figure 5.1: DynTaskMAS architecture. The DTGG decomposes tasks into a DAG; the APEE and SACMS handle parallel execution and context sharing; the AWM optimizes workflows and dispatches work to LLM-based agents.

The APEE's sole job is to execute a given graph efficiently; it does not need to understand the semantics of individual subtasks beyond their dependencies and costs. The SACMS's sole job is to move context between agents that need it; it does not need to understand what the context means beyond its semantic tags. The AWM's sole job is to observe and adjust; it does not make execution decisions directly. This separation mirrors the classical OS-kernel decomposition—process manager, scheduler, memory manager, resource monitor—each with a narrow interface and a well-defined responsibility, and it is what makes the system easier to reason about than a monolithic orchestrator.

A related architectural choice worth flagging is that the four components communicate via *explicit messages* rather than shared mutable state. When the DTGG adds a new vertex v to G, it sends a $\textsc{VertexAdded}(v, \ldots)$ message to the APEE; the APEE, upon completing v, sends a $\textsc{VertexDone}(v, \text{result})$ message to the SACMS and to the DTGG (so the DTGG can mark successors ready). This message-passing discipline is operationally heavier than shared state (every decision is serialised into a message), but it makes the components independently testable: a test harness for the APEE does not need to stand up a real DTGG; it just needs to synthesise the appropriate message sequence.

5.3 Formalisms and Scheduling Algorithm

Task Graph. The DTGG models a task as a weighted DAG G:

$$G = \langle V, E, W \rangle, \tag{5.1}$$

where $V = \{v_1, \ldots, v_m\}$ are subtask vertices, $E \subseteq V \times V$ are dependency edges, and $W : E \to \mathbb{R}^+$ assigns edge weights. Each incoming task t_i is recursively decomposed via a function $D(t_i) = \{s_{i1}, s_{i2}, \ldots, s_{ik}\}$ until atomic subtasks are reached. Edge weights encode both computational cost and communication overhead:

$$W(v_i, v_j) = \alpha \cdot C(v_j) + \beta \cdot I(v_i, v_j), \tag{5.2}$$

where $C(v_j)$ is the estimated computational complexity of subtask v_j, $I(v_i, v_j)$ is the context transfer time from v_i to v_j (managed by SACMS), and $\alpha = 1/T_c$, $\beta = 1/T_t$ are normalizing coefficients derived from average computation time per complexity unit (T_c) and average

context transfer time per information unit (T_t), respectively. The ratio α/β tends to sit between roughly one-half and two, varying with the GPU-to-network performance balance of the deployment in question.

The weighting choice deserves brief comment. Setting $\alpha = 1/T_c$ and $\beta = 1/T_t$ normalises the computational cost and the context transfer time to a common unit of wall-clock seconds. The ratio α/β then controls which factor dominates: if α/β is large (fast network, slow compute), edge weights emphasise the computational cost of the successor vertex; if small (slow network, fast compute), edge weights emphasise the context transfer time. Empirically α/β lands between 0.5 and 2.0 on commodity GPU clusters, which is wide enough that the edge-weight ordering changes non-trivially across deployments; a runtime that hard-coded a single ratio would schedule suboptimally on environments it was not tuned for.

As execution proceeds, the graph (5.1) is updated dynamically:

$$G_{t+1} = U(G_t, \Delta_t), \tag{5.3}$$

where Δ_t encodes newly discovered subtasks, completed vertices, or changed dependencies. Algorithm 5.2 details the DTGG update procedure. This continuous update mechanism is critical for handling the inherent unpredictability of LLM-based agents: an agent may discover that a subtask requires further decomposition, or that a dependency relationship was incorrectly estimated.

To handle apparent cyclic structures—common in reflection/revision loops where an agent iteratively improves its output—I impose a bounded reflection limit $N \leq 3$. Reflection terminates when: (a) the quality assessment meets a predetermined threshold, (b) the iteration count reaches N, or (c) successive improvement falls below a minimum threshold ε. This bounded-reflection mechanism prevents infinite loops while still allowing meaningful iterative refinement.

Priority Scheduling. The APEE computes task priorities over (5.1) using a critical-path-inspired formula:

$$P(v_i) = \begin{cases} C(v_i), & \text{Succ}(v_i) = \emptyset, \\ \dfrac{C(v_i)}{\max\limits_{v_j \in \text{Succ}(v_i)} (W_{ij} + P(v_j))}, & \text{otherwise}, \end{cases} \tag{5.4}$$

where $\text{Succ}(v_i)$ is the set of immediate successors. The division yields an inverse priority: tasks on long critical paths receive *lower* P values and are scheduled first.

Algorithm 5.1 shows the APEE main loop in full. The scheduler maintains a min-heap keyed on P; ready vertices (those whose predecessors are all complete) are pushed onto the heap when their last predecessor finishes, and the agent pool pops them in priority order. On completion, a vertex's result is handed to the SACMS for semantic distribution (described next), and any now-ready successors are enqueued. The heap structure and the critical-path-inspired priority together give a work-conserving scheduler that, in the sense of classical list-scheduling analyses, stays close to the optimal makespan within heuristic limits, with amortised $O(\log |V|)$ enqueue/dequeue cost.

A worked example. To make the scheduler behaviour concrete, consider the PosterAgent pipeline of Chapter 4 realised as a task graph. The Parser produces a single vertex v_0 whose output is the asset library. The Planner's asset-matching and binary-tree-layout sub-steps form two vertices v_1, v_2 depending on v_0 (and v_2 on v_1). A poster with, say, four panels then spawns four Painter–Commenter vertices v_3, v_4, v_5, v_6, each depending on v_2 but not on each other. Under Eq. (5.4) the four panel vertices have identical priorities (they are leaves with

Algorithm 5.1 APEE main loop.

Require: task graph $G = \langle V, E, W \rangle$; agent pool $\mathcal{A}$ of capacity k; priority function P as in Eq. (5.4)
Ensure: all vertices in V executed with dependencies honoured

```
heap ← min-heap keyed on P
indeg(v) ← |Pred(v)| for all v ∈ V
for each v ∈ V with indeg(v) = 0 do
    heap.PUSH(v, P(v))
end for
while heap is non-empty or A has active agents do
    while A has idle agent and heap non-empty do
        v ← heap.POP()
        dispatch v to an idle agent a ∈ A
    end while
    wait for any agent a to complete, returning (v, result)
    SACMS-DISTRIBUTE(v, result)                    ▷ push relevant updates
    for each u ∈ Succ(v) do
        indeg(u) ← indeg(u) − 1
        if indeg(u) = 0 then
            heap.PUSH(u, P(u))
        end if
    end for
end while
```

equal estimated cost), so APEE dispatches them to up to four agents concurrently. With $k = 4$ agents and panel-generation the dominant cost, the makespan drops from the serial $t_0 + t_1 + t_2 + 4t_{\text{panel}}$ to $t_0 + t_1 + t_2 + t_{\text{panel}}$, recovering a $3t_{\text{panel}}/(t_0 + t_1 + t_2 + 4t_{\text{panel}})$ fraction of the total time—consistent with the $\sim 70\%$ reduction empirically observed in Chapter 4's parallelisation experiment.

Context Distribution. The SACMS organizes context as a *context forest*:

$$\text{ContextForest} = \{T_1, T_2, \ldots, T_n\}, \tag{5.5}$$

where each tree $T_i = (V_i, E_i, r_i)$ is rooted at a top-level task r_i, and each node stores an ID, data payload, child pointers, and semantic tags.

A Semantic Analyzer processes each context update by extracting tags, entities, and relationships, constructing a semantic graph $G_s = (V_s, E_s)$ that captures the meaning of the update. The distribution manager then routes updates to agents whose tag sets are relevant, using Jaccard similarity:

$$\text{Rel}_{\text{push}}(u, a) = \frac{|ST(u) \cap ST(a)|}{|ST(u) \cup ST(a)|}, \tag{5.6}$$

where $ST(u)$ and $ST(a)$ are the semantic tag sets of the update and the agent's current context, respectively. Updates are only distributed to agents where the relevance exceeds a threshold θ_{push}, preventing information overload.

For query-time retrieval, when an agent needs specific context from the forest, cosine similarity over vector representations is used:

$$\text{Rel}_{\text{pull}}(n, q) = \cos\big(\mathbf{V}(S_n),\ \mathbf{V}(S_q)\big), \tag{5.7}$$

and an access control filter ensures that agents only receive nodes at or below their access level. Let $\mathcal{N}$ denote the set of all context nodes; the filtered result set is:

$$R = \{n \in \mathcal{N} \mid \text{Rel}_{\text{pull}}(n, q) > \theta_{\text{pull}} \ \wedge \ \text{AL}(n) \leq \text{AL}(\text{agent})\}. \tag{5.8}$$

This dual-similarity approach—Rel_{push} (Jaccard) for proactive distribution, Rel_{pull} (cosine) for on-demand queries—balances proactive information sharing with on-demand retrieval, reducing both redundant transfers and context-switching overhead.

The design choice of two different similarity functions is deliberate. Jaccard similarity operates over discrete tag sets, is cheap to compute (a set-intersection plus a set-union), and is well-suited to the proactive-push setting where the runtime must decide, for every pending context update, whether to notify each agent. Cosine similarity operates over vector embeddings of the context and the query, is more expressive but more expensive, and is well-suited to the pull setting where an agent has already decided it needs something and the runtime can afford a slower but more semantically accurate lookup. Using the same metric in both positions would trade one property for the other; the asymmetric design exploits the fact that push is latency-critical and noisy (wrong pushes waste bandwidth but do no lasting harm) while pull is correctness-critical and tolerant of latency.

A worked example again helps fix intuition. Consider the travel-planning case study of §5.4: seven domain-specialised agents (preference, destination, transportation, accommodation, attractions, culinary, itinerary) each have their own tag set (USER-PREFS, FLIGHT, HOTEL, PLACES, FOOD, SCHEDULE). When the destination agent emits a context update about Tokyo, its DESTINATION, TOKYO tags overlap strongly with the transportation, accommodation, attractions, and culinary agents (all routes to/within Tokyo matter to them), and the SACMS pushes the update to exactly those four; the preference and itinerary agents do not need it yet. Later, when the itinerary agent begins synthesising the final plan, it pulls from the context forest with a query vector encoding "Tokyo day-by-day schedule", retrieving the accommodation and attractions agents' outputs by cosine similarity regardless of whether their original tag sets included "schedule". The push and pull together achieve both timely distribution and complete retrieval, at an overall communication cost lower than broadcast.

Workflow Optimization. The AWM solves a continuous optimization problem:

$$\min_{\omega \in \Omega} \ f\big(\omega, \ M(t)\big), \tag{5.9}$$

where ω is a workflow configuration, Ω the configuration space, and $M(t)$ the current system metrics. Resource allocation follows:

$$\text{Alloc}(a_i, t) = \frac{w_i \cdot \text{Load}(a_i, t)}{\sum_j w_j \cdot \text{Load}(a_j, t)} \cdot \text{TotalRes}(t), \tag{5.10}$$

with w_i the priority weight and $\text{Load}(a_i, t)$ the current load of agent a_i.

5.4 Results

I evaluate the runtime on a cluster of four NVIDIA RTX 3090 GPUs (24GB VRAM each) with an AMD EPYC 7763 64-Core Processor and 512GB DDR4 memory, running Ubuntu 22.04 LTS with CUDA 12.1 and TensorRT-LLM [22] 0.7.1. The LLM backbone is Llama-3.1-8B [8] with INT8 quantization, batch size 32, and sequence length 2048. Table 5.1 summarizes execution time improvements across three complexity levels.

Before unpacking the findings, two methodological points are worth making explicit. First, the comparison is against a *sequential* traditional baseline: the same agents, the same

Algorithm 5.2 Dynamic Task Graph Update (DTGG).

Require: Current graph $G = \langle V, E, W \rangle$; new tasks T_{new}; changes Δ
Ensure: Updated graph G
for each $t_i \in T_{\text{new}}$ **do**
  $S_i \leftarrow \text{DECOMPOSETASK}(t_i)$ ▷ recursive
  $V \leftarrow V \cup S_i$
  for each pair $(s_{ij}, s_{ik}) \in S_i$ where s_{ik} depends on s_{ij} **do**
    $E \leftarrow E \cup \{(s_{ij}, s_{ik})\}$
    $W(s_{ij}, s_{ik}) \leftarrow \alpha \cdot C(s_{ik}) + \beta \cdot I(s_{ij}, s_{ik})$
  end for
end for
for each change $\delta \in \Delta$ **do**
  $G \leftarrow \text{APPLYCHANGE}(G, \delta)$
end for
return G

Table 5.1: DynTaskMAS performance: execution time and scalability.

Complexity	Traditional (s)	DynTaskMAS (s)	Improv.
Simple (5–10)	4.7 ± 0.3	3.7 ± 0.2	21.3%
Medium (20–30)	9.8 ± 0.5	7.1 ± 0.3	27.6%
Complex (50+)	18.5 ± 0.8	12.4 ± 0.5	33.0%
Scalability (throughput in tasks/s):			
4 agents		12.3 ± 0.4	—
8 agents		23.1 ± 0.6	1.88×
16 agents		42.7 ± 0.9	3.47×
32 agents		76.4 ± 1.2	6.21×

task graphs, but executed one vertex at a time without the APEE parallelisation or the SACMS context routing. This is a deliberate choice of baseline, because it isolates the contribution of the runtime layer from the contribution of the agents' underlying capabilities. Second, the three complexity levels (simple, medium, complex) correspond to task graphs of roughly 5–10, 20–30, and 50+ subtasks respectively, with edge density held approximately constant; this allows us to see how the runtime's advantage *scales with task complexity* rather than conflating complexity and size.

Three findings emerge from the experiments.

The first is that the runtime's advantage grows with task complexity. On simple graphs of 5–10 subtasks the wall-clock reduction is 21.3%; on medium graphs it climbs to 27.6%; on complex graphs of 50+ subtasks it reaches 33.0%. The trend is consistent with the qualitative argument the graph-dependency analysis already suggested: larger graphs contain more structurally-independent branches, more context-update overlap, and therefore more room for each of the three components (DTGG, APEE, SACMS) to act. On a purely serial dependency chain the runtime's gains would be close to zero; the tasks here have enough latent parallelism that the gains are material.

The second is that the runtime improves resource utilisation substantially on a more realistic workload. In a seven-agent travel-planning case study—with agents for user preferences, destinations, transport, accommodation, attractions, culinary choice, and itinerary synthesis—GPU utilisation rises from 65% to 88%, agent coordination overhead drops from 850 ms to 320 ms, and the number of context switches falls from 42 to 18. Table 5.2 summarises these numbers. The improvements are large enough to matter for practical deploy-

Table 5.2: Travel-planning case study: seven domain-specialized agents, before and after DynTaskMAS.

Metric	Baseline	DynTaskMAS	Change
Resource utilization	65%	88%	+23 pp
Agent coordination time	850 ms	320 ms	−62%
Context switches	42	18	−57%

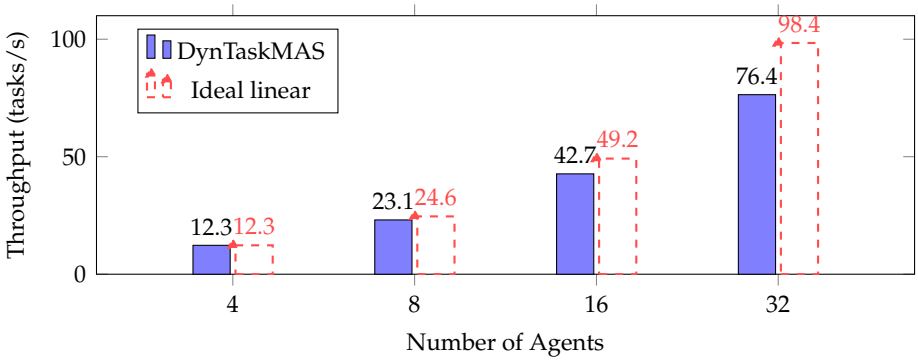

Figure 5.2: DynTaskMAS throughput scaling. The runtime achieves near-linear scaling up to 16 agents (3.47× for 4× agents) with sub-linear returns at 32 agents due to SACMS contention and scheduling overhead.

ment, and they come from the same three components that produced the throughput gains in the synthetic benchmark.

The third is that scaling is near-linear up to sixteen agents and then turns sub-linear. Moving from four to sixteen agents (a 4× capacity increase) yields a 3.47× throughput increase, an 87% scaling efficiency. At thirty-two agents throughput hits 76.4 tasks per second—6.21× the four-agent baseline but only 1.79× the sixteen-agent figure. The efficiency loss at the largest scale has two sources, both of them structural rather than transient: contention on shared SACMS state when many agents query it simultaneously, and the per-dispatch cost the APEE pays to keep the priority heap correct.
Figure 5.2 visualizes the throughput scaling behaviour, with the dashed line indicating ideal linear scaling for reference. Run-to-run variance is small (±0.2–0.5 s for execution time, ±0.4–1.2 tasks/s for throughput), and the coefficient of variation actually shrinks as graphs grow larger—an unexpected but welcome property that suggests the runtime's aggregate behaviour is easier to predict on realistic workloads than on toy ones.

5.5 Discussion

An important architectural property of my runtime is that it is *agnostic to the programming model*: the DTGG consumes task descriptions and dependency specifications, not framework-specific agent definitions. This means that DynTaskMAS could, in principle, schedule workflows authored in the conversation-programming paradigm of Chapter 3 or execute PosterAgent's Parser–Planner–Painter–Commenter pipeline (Chapter 4) as a dependency DAG.

From a systems perspective, DynTaskMAS occupies the same role as an operating-system scheduler in the classical computing stack. Its task graph formalism is analogous to a process dependency graph; the APEE's priority scheduling uses critical-path-based prioritization; and the SACMS's semantic routing is analogous to a distributed shared-memory system with tag-based coherence. The key innovation is adapting these classical concepts to

the unique characteristics of LLM-based agents: non-deterministic execution times, dynamically evolving task structures, and semantic (rather than address-based) context sharing.

Three specific differences from a traditional OS scheduler are worth highlighting. First, the *unit of work* is an agent reply rather than a CPU instruction; replies vary in duration by orders of magnitude (a fast tool call in milliseconds vs. a slow LLM generation in seconds), and the scheduler must be robust to this variance rather than amortising small deterministic tasks. Second, the *dependency graph is dynamic*: a classical OS scheduler receives a fixed process graph and orchestrates execution, whereas the DTGG edits the graph as new subtasks are discovered, which rules out static scheduling algorithms and requires the priority-heap-based online approach I adopt. Third, the *coherence model is semantic*: a classical distributed shared-memory system decides what to cache based on address locality; the SACMS decides based on content similarity. The shift from address to content is what makes tag-based and vector-similarity filters useful where traditional locality-based coherence would not apply.

However, the runtime assumes that task descriptions and dependencies are provided as inputs—it does not extract them from high-level agent programs. This gap between programming models and runtime scheduling is precisely where a compilation layer (discussed in Chapter 6) would be most valuable. A further limitation is that my evaluation relies on synthetic benchmarks—a three-complexity-level microbenchmark (Table 5.1) and a seven-agent travel-planning case study (Table 5.2)—rather than established multi-agent benchmarks with ground-truth solutions, making it difficult to disentangle scheduling gains from task-specific variance. Additionally, the bounded-reflection mechanism ($N \leq 3$) for cyclic dependencies is a pragmatic heuristic rather than a principled convergence guarantee; for safety-critical applications, formal termination proofs would be preferable.

Where the Runtime's Advantage Comes From

The 21–33% improvements and near-linear scaling reported above are the aggregate effect of three component-level contributions that are worth disentangling.

DTGG-side: unlocking latent parallelism. A traditional sequential executor processes vertices in a topological order, using at most one agent at any instant. The DTGG's explicit dependency graph lets the APEE dispatch any vertex whose predecessors are complete, which—for task graphs with significant width—immediately converts dependency slack into throughput. In the three-complexity-level experiments, the complex (50+ subtask) graphs have the highest width, which is precisely why their improvement (33%) exceeds the simple (5–10 subtask) graphs' (21%). Width, not depth, is what the DTGG unlocks.

APEE-side: priority-guided work-stealing. Even given parallelism, a naïve scheduler might leave the critical path idle while running off-path work. The priority heap (Eq. 5.4) keeps the longest-successor chain running first, ensuring that the critical path is never waiting for an off-path vertex. On the seven-agent travel example, this shows up as the utilisation jump from 65% to 88%: the 23-percentage-point improvement is almost entirely attributable to priority scheduling, because all vertices on the critical path of an itinerary—preferences, flights, hotels, itinerary synthesis—are now never starved by lower- priority leaf tasks.

SACMS-side: avoiding context stalls. Even given parallelism and priority, a runtime that broadcasts every context update to every agent will stall on the communication cost of those updates. The Jaccard-gated push path reduces that cost by only broadcasting updates the receiving agent is demonstrably interested in; the cosine-based pull path eliminates a different cost, the latency of repeated broadcast when an agent retroactively realises it needs something. Context-switch count drops from 42 to 18 (57%) in the travel example for this reason, and the coordination-time reduction from 850 ms to 320 ms (62%) largely follows from fewer switches plus smaller per-switch payloads.

Multiplying the three: parallelism exposure times priority guidance times communication reduction yields the observed throughput numbers. Removing any single component would drop a meaningful fraction of the gain, which explains why none of the three components is a straightforward port from classical distributed computing—each needed adaptation to the specific properties of LLM-based execution.

5.6 Summary

In this chapter I introduced DynTaskMAS, a runtime that turns a multi-agent workflow into a dynamic directed acyclic graph, schedules subtasks across parallel agents with priority-based queuing, and manages shared context through semantic-aware distribution. The evaluation demonstrated 21–33% execution-time reductions over sequential execution and near-linear scaling up to 16 concurrent agents. DynTaskMAS provides the execution substrate on which the programming model of Chapter 3 and the workloads of Chapter 4 can run efficiently.

Chapter 6

Cross-Layer Synthesis and Open Challenges

Having examined each layer individually, I now step back to analyze how the three layers connect, what synergies emerge when they are composed, and what fundamental challenges arise at their interfaces. This cross-layer perspective is the central analytical contribution of this thesis: it reveals connections that are invisible when each layer is studied in isolation and surfaces concrete research directions that require cross-layer co-design.

6.1 Connecting the Layers

Table 6.1 maps key system concepts across the three layers, and Figure 6.1 complements the table with a *data-flow* view: it shows which artifact from one layer becomes which artifact in the next, and where an interface is missing today. I analyze each inter-layer connection in detail.

Layer 1 → Layer 2 (Programming → Workload). The `ConversableAgent` abstraction of Chapter 3 could directly implement the three roles of PosterAgent (Chapter 4): the Parser and Planner as `AssistantAgent` instances backed by tool functions (Marker/Docling for parsing, binary-tree layout for planning), and the Painter–Commenter as a nested two-agent conversation where a rendering agent and a VLM-based critic communicate via the auto-reply loop, with `register_reply()` implementing the termination condition. This mapping demonstrates that AutoGen's abstractions are expressive enough to encode PosterAgent's pipeline, but also reveals what is missing: AutoGen provides no mechanism to declare that the Parser must complete before the Planner, or that individual panels are independent—such dependencies are implicit in the conversation structure rather than explicit in a schedulable representation.

Layer 2 → Layer 3 (Workload → Runtime). PosterAgent's pipeline is inherently a DAG: the Parser must complete before the Planner, and the Planner before the Painter, but *within* each stage, panel-level operations are independent. DynTaskMAS's DTGG could decompose this into a task graph with three sequential phases and within-phase parallelism. For example, if a poster has 8 panels, the APEE could dispatch 8 concurrent Painter–Commenter tasks, each assigned to a different agent, while the SACMS routes only the relevant figure assets and section synopses to each agent via Jaccard similarity filtering. The parallelization experiment in Chapter 4 validates this potential: parallelizing content generation reduces time by 69.3% (from 176.69s to 54.16s), illustrating the same class of gains—exploiting independent subtasks—that DynTaskMAS targets at the runtime level (21–33% reduction across varying complexity).

Layer 1 → Layer 3 (Programming → Runtime). AutoGen's conversation events (send, receive, generate_reply) could serve as the *scheduling primitives* consumed by DynTaskMAS. Each `generate_reply()` call represents a schedulable unit of work; the auto-reply mechanism naturally generates dependency edges (an agent's reply depends on the message it received). A *compilation step* that statically analyzes AutoGen conversation programs— identifying which reply chains are independent, which require sequential ordering, and which involve shared context—could automatically produce DynTaskMAS-compatible task graphs. This would close the loop from programming to efficient execution, allowing developers to write natural conversation programs in AutoGen while benefiting from DynTaskMAS's parallel scheduling without manual DAG construction.

Table 6.1: Cross-layer concept mapping.

Concept	Layer 1 (AutoGen)	Layer 2 (Paper2Poster)	Layer 3 (DynTaskMAS)
Abstraction	ConversableAgent	Parser / Planner / Painter–Comm.	Task vertex v_i
Coordination	send/receive, auto-reply loop	Painter–Comm. loop	SACMS context dist.
Control Flow	NL + code + hybrid	Pipeline stages	DAG edges + priority sched.
Parallelism	Not explicit	Panel-level (69.3% time reduction)	APEE (3.47× at 16 agents)
Evaluation	Task-specific accuracy	4-dim. framework	Exec. time, utilization

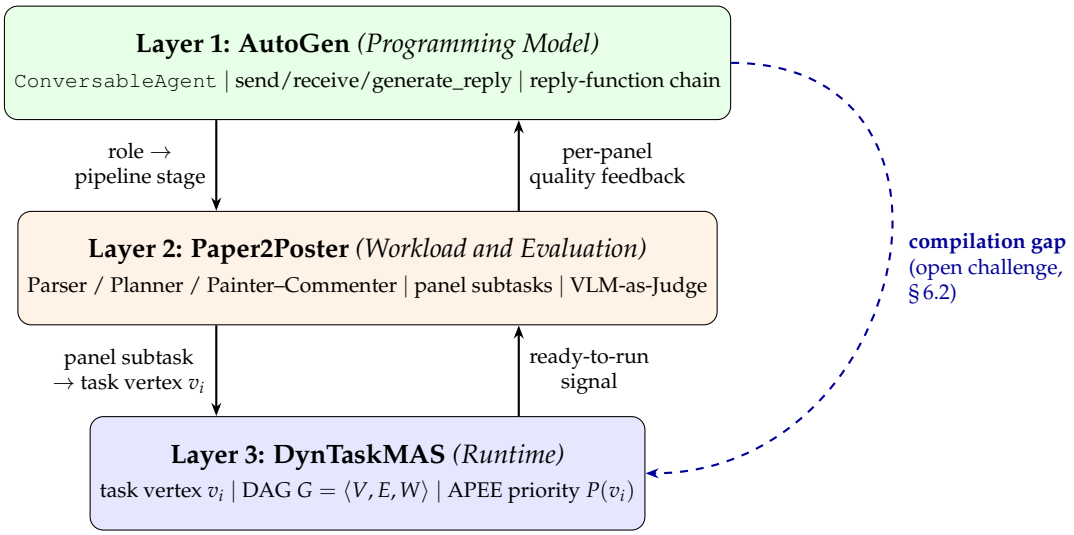

Figure 6.1: Concrete data flows across the three layers. Roles defined in Layer 1 (`ConversableAgent` subclasses) instantiate Layer 2's pipeline stages; Layer 2's panel-level subtasks become Layer 3's task vertices v_i inside the DAG G. Ready-to-run signals flow back up from the runtime, and per-panel quality feedback from the evaluation framework closes the loop to the programming model. The dashed curve on the right marks the Layer 1→Layer 3 *compilation gap*: no existing tool extracts the implicit task DAG from a conversation program and feeds it directly to the runtime.

6.2 Open Challenges

1) Unified Programming-to-Runtime Compilation. None of the layers, as currently designed, provides an end-to-end path from high-level agent definitions (Layer 1) to optimized parallel execution (Layer 3). A *compiler* that analyzes AutoGen conversation programs, extracts the implicit task DAG, and feeds it to a DynTaskMAS-style scheduler would significantly reduce the engineering effort required for efficient deployment.

Concretely, the compiler I have in mind would operate in three passes. A first *agent-graph-extraction* pass walks the `register_reply()` chain of every `ConversableAgent` in the user's program and constructs a static skeleton of possible conversation topologies. A second *dependency-inference* pass annotates each reply function with a conservative read/write set over shared-context keys (derived from type hints or a light static analysis of the function body). A third *DAG-synthesis* pass fuses the skeleton and the dependency

annotations into a DynTaskMAS-compatible task graph $G = \langle V, E, W \rangle$ where each reply invocation is a vertex, static dependencies become edges with weights set by the read/write overlap, and branches of the skeleton that cannot be statically resolved become speculative subgraphs guarded by a runtime predicate.

The challenges are non-trivial: AutoGen programs involve dynamic control flow (LLM-proposed function calls, conditional human input) that the first two passes cannot fully resolve. Hybrid approaches combining static analysis with speculative execution—analogous to branch prediction in CPU pipelines—could offer a practical middle ground: compile the statically-resolvable portion of the graph, leave a small set of speculative branches as dynamic guards, and let the runtime commit or discard branches as LLM decisions arrive. A prototype could bootstrap from the observation that in practice most AutoGen programs have only a handful of branching points, most of which are `GroupChatManager` speaker selections that admit finite enumeration.

2) Cross-Workload Evaluation Standards. The four-dimensional framework of Chapter 4 is tailored to poster generation. Generalising such multi-dimensional evaluation—combining task-specific accuracy, output quality, computational cost, and information density—to other MAS workloads (code generation, scientific discovery, embodied reasoning) remains an open problem. A meta-framework that defines evaluation *dimensions* (rather than metrics) and provides composable building blocks for domain-specific instantiation would be a valuable community contribution.

The PaperQuiz methodology is particularly promising as a transferable pattern: the idea of generating questions from reference material and using multiple blind readers to assess whether the generated artifact contains the reference's information is not specific to posters. For code-generation workloads, a "code comprehension quiz" could ask: given the generated code, can a reader reconstruct the specification? For embodied-reasoning workloads, a "plan trace quiz" could ask: given the generated plan, can a reader answer questions about what the agent will do in each state? The shape of the evaluation transfers cleanly; the work is in designing the question-generation prompt for each domain.

3) Cost-Aware Scheduling. DynTaskMAS optimizes execution time and resource utilization, but in practice a major cost driver for LLM-based MAS is often API pricing rather than compute time. The cost analysis of Chapter 4—ranging from \$0.005 for Qwen to \$1.87 for OWL-4o per poster, a 374× difference—underscores the need for cost-aware scheduling. A runtime that jointly optimizes latency, quality, and monetary expenditure could, for example, route simple subtasks to cheaper models (Qwen-2.5-7B) while reserving expensive models (GPT-4o) for quality-critical tasks like final poster assessment. This requires the runtime to maintain a cost model for each LLM backend and a quality estimator for each task type—capabilities that my current runtime and most existing runtimes do not yet offer.

The extension I envision treats each task vertex v_i as parameterised by a *backend choice* $b(v_i) \in \mathcal{B}$ and an *expected quality* $q(v_i, b)$ that depends on both the vertex and the chosen backend. The APEE then solves a constrained optimisation per vertex: $\min_b \text{cost}(v_i, b)$ subject to $q(v_i, b) \geq q_{\min}(v_i)$, where $q_{\min}$ is a per-vertex quality floor derived from the workload specification. Estimating $q(v_i, b)$ is the hard part—it requires either a cost-conditional quality oracle (trained offline) or a bandit-style online learner that refines estimates from observed outcomes. The Paper2Poster experiments already provide useful offline training data: each (method, cost, quality) triple in Table 4.2 is a point from which a crude backend-quality model can be fit.

4) Safety and Verification. The OptiGuide application in Chapter 3 shows that multi-agent safety guards improve unsafe-code detection, but systematic verification across the stack remains unsolved. Three concrete concerns deserve treatment. First, *runtime parallelisation* may reorder interactions in ways that violate safety invariants; a Safeguard agent that

checks a code snippet for unsafe operations before execution cannot protect against a race in which a separate agent invokes a related tool concurrently. Second, *context routing* may expose sensitive information to agents that should not see it, especially when the semantic similarity heuristic routes a context update to an unexpected subset of agents. Third, *dynamic task-graph updates*—my runtime's DTGG continuously updates G as new subtasks are discovered—may introduce subtasks that bypass checks set up for the original graph.

A principled approach would adapt concurrent program verification to conversation programs, treating each agent as a thread, each reply function as an atomic action (modulo its read/write set), and each context update as a shared-memory write. Existing work on commutativity-based verification of transactional memory and safe speculative execution in hardware pipelines both offer transferable ideas.

5) Adaptive Model Selection. All three layers expose the trade-off between model capability and cost. The programming model of Chapter 3 uses GPT-4 in its examples without guidance on when cheaper models suffice; Chapter 4 shows a 7B model achieves near-GPT-4o quality through pipeline design; Chapter 5 schedules agents without considering backend heterogeneity. An integrated stack would support *adaptive model selection*, dynamically routing subtasks to the most cost-effective model based on difficulty and quality requirements.

Two observations make this feasible in the near term. First, the PosterAgent pipeline already demonstrates that a *fixed* model assignment per pipeline stage can be competitive: the Parser, Planner, and Painter roles each have stable quality requirements, and GPT-4o is overkill for some (e.g., the Planner's asset-matching subtask) while Qwen-2.5-7B is underpowered for others. A per-role assignment that used these profiles as priors would close much of the cost gap without any online learning. Second, the SACMS's per-vertex cost and latency telemetry (which the runtime collects for load-balancing) is exactly the signal an adaptive selector needs, so the infrastructural cost of adding selection is modest: a single post-execution record per vertex tagged with the backend chosen and the observed outcome.

Challenge 5 is deliberately listed last because it is the most *cross-cutting* of the five: it touches the programming-model layer (annotations on reply functions describing quality requirements), the workload layer (calibration data from the four-dimensional evaluation), and the runtime layer (the selector itself). A clean solution would require co-design across all three.

Why Five Challenges, Not More or Fewer

The five challenges above are not an exhaustive list of everything that could be improved across the stack. I selected them by a specific criterion: each is a problem whose solution requires co-design across at least two layers and which is not adequately addressed by any single-layer contribution. Problems that can be solved by improving one layer in isolation—faster LLM inference, better retrieval augmentation, better CLIP embeddings—are legitimate research questions, but they are not the questions this thesis is positioned to illuminate.

Conversely, five rather than three or four: any smaller list would have either omitted a genuinely cross-layer concern (safety and evaluation both deserve their own entries) or forced two distinct open problems into the same bullet. The five chosen are structurally distinct: compilation connects Layers 1 and 3; evaluation standards and cost-aware scheduling connect Layer 2 with Layers 1 and 3 respectively; safety spans all three; and adaptive model selection is a cross-cutting quality/cost negotiation. Each has a distinct technical shape and a distinct set of candidate solutions.

6.3 Summary

This chapter showed that the three layers of my agentic systems stack are not independent: information flows across interfaces, and choices at one layer constrain the options available at others. The Layer 1→Layer 3 compilation pathway emerges as the principal open interface, and the four further open challenges identified here (five in total, with compilation counted) motivate much of the future work discussed in Chapter 7.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This thesis has proposed a three-layer *agentic systems stack*— programming model, workload and evaluation, and runtime—as a unifying lens for understanding the lifecycle of LLM-based multi-agent systems. Each layer addresses a distinct systems concern and contributes unique insights.

At the **programming-model layer** (Chapter 3), I showed that *conversation can serve as a versatile programming primitive*, unifying LLMs, humans, and tools under a single `ConversableAgent` abstraction that supports applications from mathematical reasoning to supply-chain optimization. The `send/receive/generate_reply` interface, the programmable auto-reply loop, and the modular `register_reply()` mechanism together replace rigid hand-coded orchestration with a flexible conversation-programming workflow. Six application case studies—spanning two-agent math solving, a retrieval-augmented question-answering system, a three-agent embodied-planning configuration, the OptiGuide supply-chain assistant, a four-agent group chat for complex coding, and a natural-language chess interface—demonstrate the range of conversation patterns the abstraction accommodates without any fundamental framework modification. The OptiGuide line-count reduction (430 lines to 100 lines with 4.3× boilerplate savings) is particularly telling: conversation programming is not a toy abstraction; it is a genuinely productive way to structure real multi-agent code.

At the **workload-and-evaluation layer** (Chapter 4), I established that realistic multi-agent tasks demand multi-modal, long-context processing with extreme compression (14.4×) and multi-dimensional evaluation. The PosterAgent pipeline achieves near-human quality (3.72 vs. 3.77 VLM-Judge) while demonstrating that pipeline design can outweigh model scale ($0.005 per poster with Qwen-2.5-7B). The four-dimensional evaluation framework—CLIP-based visual metrics, perplexity-based textual coherence, VLM-as-Judge holistic assessment, and the novel PaperQuiz information-transfer probe—provides a transferable template for multi-dimensional workload evaluation. The per-baseline failure analysis in Chapter 4 clarified precisely why each baseline lagged: 4o-Image lacks iterative refinement and generates visually attractive but textually noisy posters; OWL-4o pays heavily in token budget for autonomous exploration; PPTAgent applies slide-generation heuristics that are wrong for the poster canvas; 4o-HTML competes closely but is held back by a Commenter without the in-context reference trick. Each of these failure modes maps onto a design choice in my pipeline, and seeing the failures from the outside clarifies which choices are load-bearing.

At the **runtime layer** (Chapter 5), I demonstrated that classical scheduling concepts—DAG analysis, critical-path prioritization, context-aware routing—adapt effectively to LLM-based agents. DynTaskMAS reduces execution time by 21–33% with near-linear scaling to 16 concurrent agents, all while managing shared context through a Jaccard-plus-cosine semantic routing scheme that balances proactive distribution against on-demand retrieval. The scheduler's central design choice—treating each `generate_reply` call as a task vertex rather than a conversation turn—is what unlocks the parallelism: vertices whose dependency edges do not reach each other are free to run concurrently, and the APEE's priority-heap loop keeps the critical path short without sacrificing throughput. The travel-planning case study (utilisation from 65% to 88%, coordination time from 850 ms to 320 ms, context

switches from 42 to 18) shows that the gains translate from synthetic micro-benchmarks into more realistic seven-agent workloads.

Finally, the **cross-layer synthesis** of Chapter 6 revealed that the three layers are not independent: conversation-programming events can be compiled into runtime task graphs, workload decomposition dictates achievable parallelism, and evaluation frameworks must eventually price in the computational cost that a runtime incurs. That synthesis articulated five open challenges—programming-to-runtime compilation, cross-workload evaluation standards, cost-aware scheduling, safety verification, and adaptive model selection—which scope the lines of future work below.

The three layer-specific contributions (C1–C3) supply a programming model, a workload-and-evaluation framework, and a runtime—one for each layer of the stack. The fourth contribution (C4), the cross-layer synthesis of Chapter 6, identifies the missing compilation pathway from Layer 1 conversation programs to Layer 3 task graphs as the single change that would most dramatically reduce the engineering effort required to build and deploy multi-agent applications at scale. The next section sketches how that pathway, together with four further open problems, could be closed.

7.2 Future Work

The cross-layer synthesis of Chapter 6 surfaced five open challenges. I close this thesis by sketching four immediate research lines: three that take up the corresponding challenges from § 6.2 directly—unified programming-to-runtime compilation (§ 6.2 challenge 1), cross-workload evaluation standards together with cost-aware scheduling (§ 6.2 challenges 2 and 3), and safety and verification (§ 6.2 challenge 4)—plus one cross-cutting direction on human oversight that spans all three layers. The fifth challenge, adaptive model selection, is a natural extension of the cost-aware evaluation thread and is left for follow-up work.

Unified Programming-to-Runtime Compilation. Conversation-programmed agents today emit messages without any awareness of the runtime's scheduling policy, and the runtime receives no structured task graph extracted from the program. Closing this gap in both directions—a static analysis pass that walks the `register_reply()` chain of each `ConversableAgent` and emits a conservative dependency specification for the DTGG, paired with light programming-model annotations (e.g., "this reply is latency-critical" or "this tool call may be parallelized with siblings") that flow back to the priority queue—would let developers write natural conversation programs and still benefit from parallel execution.

A realistic first step is a *profile-guided* compiler extension that records, for a small number of training runs, which reply chains actually fire and under what branch conditions. The resulting execution trace then becomes the scaffolding for subsequent compilation: frequent branches compile into concrete subgraphs; rare branches remain speculative. This echoes the evolution of CPU profile-guided optimisation in the late 1990s, which adopted exactly this bootstrap strategy before static branch predictors matured. For AutoGen-style systems, the analogous move is available today: the runtime already sees every reply, so harvesting that telemetry costs essentially nothing beyond the logging infrastructure.

Cross-Workload Evaluation Standards and Cost-Aware Scheduling. The PaperQuiz probe and the other three evaluation dimensions of Chapter 4 measure quality but not the economic cost of achieving it, and they are tailored to one workload. A cost-aware companion metric (tokens/USD/second per unit of quality) plus a generalisation of the four-dimensional framework to other workloads (code review, embodied decision making,

scientific discovery) would let the runtime jointly optimise latency, quality, and monetary expenditure—routing simple subtasks to cheaper models while reserving expensive models for quality-critical tasks.

Safety and Verification. The SACMS in Chapter 5 currently trusts all agents uniformly, and the runtime gives no formal guarantees about the safety invariants of parallelised execution. A version that enforces per-agent context visibility—through classical label-based access control or a policy engine composed with the semantic-similarity filter—and a verification layer that proves serialisability under runtime reordering would close a real gap in confidential multi-agent use cases, especially in enterprise settings where different agents operate under different trust levels.

A promising design is to extend the context-forest's node metadata with a security lattice label alongside the existing access-level integer, so that a context update tagged as confidential cannot be distributed to an agent whose label is below the node's label even if the Jaccard similarity is above θ_{push}. The serialisability guarantee can be obtained by treating the dependency graph $G = \langle V, E, W \rangle$ as a partial order and requiring that the observed execution schedule be consistent with some topological sort of G; this is a standard result in transactional-memory verification and transfers cleanly once a sound read/write-set annotation is available on each reply function.

Typed Message Protocols Alongside the Natural-Language Channel. The programming-model limitation highlighted in §3.5—that serialising structured data through natural-language messages imposes a per-exchange tax in both tokens and reliability—suggests a research direction complementary to compilation. A future version of my framework would expose an *orthogonal* typed channel alongside the natural-language one, so that a parser-to-planner handoff of a structured asset library (or a tensor handoff between model-inference agents) can travel as a typed payload with compile-time checks, while natural-language coordination continues to ride on the existing message channel. The challenge is keeping the interface simple enough that users do not have to declare schemas for every message; one promising route is to infer schemas from the argument annotations of registered reply functions, reusing existing Python type hints. This line connects naturally with the compilation-pathway challenge above: a typed payload is exactly the kind of information a compiler could use to derive precise read/write-set annotations for the DAG.

Human-in-the-Loop at Each Layer. Finally, each of the three layers admits a different flavour of human oversight: programming-model humans set intent, workload-level humans adjudicate quality, and runtime-level humans can interrupt or redirect running agents. A unifying account of human-in-the-loop across the stack—with a common vocabulary for intervention points and a principled treatment of latency versus autonomy—is, to my knowledge, still missing, and could dovetail naturally with the unified programming-to-runtime compilation challenge listed first.

The latency-versus-autonomy tension deserves separate study. A human-in-the-loop intervention point is, by definition, a synchronous wait on a slow channel—humans respond in seconds to minutes, not milliseconds. In a parallel-scheduled multi-agent system, the cost of that wait is amplified: every dependent subtask downstream of the intervention point is blocked, and the agent pool sits partly idle while a human contemplates. The present thesis has nothing to say about when such pauses are acceptable, how they should be amortised across speculative execution, or how the runtime should communicate uncertainty to the human so that their decision is quick. All three are open, cross-layer questions that the current form of the stack cannot resolve in isolation.

Closing each of these edges would carry the agentic systems stack from a useful organising lens—the contribution of this thesis—into the shared substrate on which future multi-agent applications are written, measured, and run.

Bibliography

[1] Anthropic. The Claude 3 model family: Opus, sonnet, haiku. *Anthropic Technical Report*, 2024.

[2] Shuai Bai, Keqin Chen, et al. Qwen2.5-VL: Scaling vision-language models for perception and understanding. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Steve Canny. python-pptx: Generate and manipulate open XML PowerPoint files. *GitHub repository*, 2023. `https://github.com/scanny/python-pptx`.

[4] Harrison Chase. LangChain. *GitHub repository*, 2022. `https://github.com/langchain-ai/langchain`.

[5] Zhongzhi Chen, Guang Liu, Bo-Wen Zhang, Fulong Ye, Qinghong Yang, and Ledell Wu. AltCLIP: Altering the language encoder in CLIP for extended language capabilities. *arXiv preprint arXiv:2211.06679*, 2022.

[6] Chroma. Chroma: The AI-native open-source embedding database. *GitHub repository*, 2022. `https://github.com/chroma-core/chroma`.

[7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proc. International Conference on Machine Learning (ICML)*, 2024.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[9] Taicheng Guo, Xiuying Chen, Yaqi Wang, et al. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

[10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[11] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.

[12] Mengkang Hu, Yuhang Zhou, Wendong Fan, Yuzhou Nie, Bowei Xia, Tao Sun, Ziyu Ye, Zhaoxuan Jin, Yingru Li, Zeyu Zhang, Yifeng Wang, Qianshuo Ye, Ping Luo, and Guohao Li. OWL: Optimized workforce learning for general multi-agent assistance in real-world task automation. *GitHub repository*, 2025. `https://github.com/camel-ai/owl`.

[13] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[14] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, and Kenton Lee. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

[15] LangChain. LangGraph: Build resilient language agents as graphs. *GitHub repository*, 2024. https://github.com/langchain-ai/langgraph.

[16] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. OptiGuide: Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023.

[17] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

[19] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, et al. Docling: An efficient open-source toolkit for AI-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025.

[20] Microsoft. AutoGen 0.4: A programming framework for agentic AI. *GitHub repository*, 2024. https://github.com/microsoft/autogen.

[21] Yohei Nakajima. BabyAGI. *GitHub repository*, 2023. https://github.com/yoheinakajima/babyagi.

[22] NVIDIA. TensorRT-LLM: A framework for optimizing large language model inference. *GitHub repository*, 2024. https://github.com/NVIDIA/TensorRT-LLM.

[23] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[24] Vik Paruchuri. Marker: Convert PDF to markdown + JSON quickly with high accuracy. *GitHub repository*, 2025. https://github.com/VikParuchuri/marker.

[25] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. *arXiv preprint arXiv:2103.00020*, 2021.

[26] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[27] Rohit Saxena, Pasquale Minervini, and Frank Keller. PosterSum: A multimodal benchmark for scientific poster summarization. *arXiv preprint arXiv:2502.17540*, 2025.

[28] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[30] Hugo Touvron, Louis Martin, Kevin Stone, et al. LLaMA 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[31] Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[33] Hao Zheng, Xinyan Guan, Hao Kong, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Ben He, Xianpei Han, and Le Sun. PPTAgent: Generating and evaluating presentations beyond text-to-slides. *arXiv preprint arXiv:2501.03936*, 2025.